# Summary of Book3  Sans 401

**The Need for Vulnerability Assessment at Scale**
The Main Idea:

Big companies have thousands (or even millions) of computers, servers, and devices. Each of these could have security weaknesses (vulnerabilities) — and new ones appear every single day.

So, the company needs a way to find and fix these weaknesses automatically and at scale — not just manually checking each device one by one.

That's why they need a vulnerability assessment and management program that can work on a large scale.

Breaking It Down:

Too Many Systems to Track

Large organizations have so many systems that they often don't even know where everything is.

They may not have a clear list of all their devices or who owns them.

Without that, it's hard to know what to secure or patch.

New Weaknesses Every Day

Hackers and security researchers find new vulnerabilities daily.

That means the number of things that need fixing keeps growing fast.

Most Breaches Start with an Unpatched System

Many big cyberattacks happened because a company didn't patch (update) one vulnerable system.

Even though patching seems simple, doing it across thousands of systems is difficult.

Not Everything Can Be Patched

Some systems are too old.

Some are vendor-controlled or too critical to take offline.

Some patches might not even work correctly.

The Real Challenge: Doing It at Scale

Scanning and fixing a few computers is easy.

Doing that for 10,000+ systems is extremely hard.

That's why automation and good planning are essential.

The Goal

Create a consistent, automated, and prioritized process to:

Find vulnerabilities

Decide which ones are most dangerous

Fix the most important ones first

In Simple Terms:

You can't protect what you don't know you have.
Big companies must use automated tools to scan all their devices, find weaknesses, and fix the most dangerous ones first — because doing it manually is impossible.

**Definitions**
**Vulnerability** : Flaw or weakness in a system that can be exploited

A security audit is like a checkup for your security — just like a doctor's checkup, but for computers and systems.

It's done to:

Assess the adequacy of controls →
See if the security measures (like passwords, firewalls, access rules, etc.) are strong enough to protect the system.

Evaluate compliance →
Check if the organization is following the rules, laws, and policies it's supposed to (like ISO, GDPR, or company security policies).

Example:

Imagine your company says,

“All employees must use strong passwords and change them every 90 days.”

A security audit will:

Check if everyone is actually doing that.

See if the password system works properly.

Report problems and suggest improvements.

**Vulnerability Assessment** : Description and analysis of vulnerabilities in a system

**Vulnerability Management (simple meaning)**

Vulnerability management means continuously finding and dealing with security weaknesses in your systems.

It’s not something you do once — it’s an ongoing, repeated process that helps keep systems safe over time.

It Involves Three Main Steps:

Identifying –
Find weaknesses (vulnerabilities) using scanners or tests.
→ Example: “This server has an old version of software that hackers can exploit.”

Remediating –
Fix the problem.
→ Example: Install a patch or update the software.

Accepting Risk –
Sometimes you can’t fix it (no patch available, or it might break something important).
→ In that case, the company decides to accept the risk but keeps an eye on it.

In short:

Vulnerability management is a continuous process of finding, fixing, and managing security weaknesses to reduce risk and keep systems secure.

**CVSS stands for Common Vulnerability Scoring System**.
Simple Explanation:

CVSS is a standard way to measure how serious a security vulnerability is.
It gives each vulnerability a score from 0 to 10, where:

Why it's useful:

There are thousands of vulnerabilities, so security teams use the CVSS score to decide which ones to fix first.
Higher scores = higher risk = higher priority.

Example:

If a web server has two vulnerabilities:

One has a CVSS score of 9.8 (can be exploited remotely)

Another has a score of 4.3 (needs local access)

→ The team will fix the 9.8 one first because it's critical.

In short:

CVSS is a system that gives each security weakness a numerical score (0–10) to show how dangerous it

**What is the VAF?**

The Vulnerability Assessment Framework (VAF) is a step-by-step process used to find, understand, fix, and report security weaknesses in a system or network.

Think of it like a security checkup plan — you follow certain stages to make sure nothing important is missed.

Key Idea

Each phase has its own inputs and results.
You must analyze the data from one step before moving on to the next one.

Different organizations (like NIST, PTES, or OSSTMM) have their own versions, but most follow the same basic 7 steps.

The 7 Phases of the Vulnerability Assessment Framework

Engagement Planning
→ Decide what will be tested, who's involved, what tools to use, and what the goals are.
(Like planning your security checkup.)

Threat Modeling
→ Identify what could go wrong — what assets are valuable, who might attack them, and how.
(Think: what are the risks and who might exploit them?)

Discovery
→ Find all systems, applications, and devices in the scope.
(You can't protect what you don't know exists.)

Scanning
→ Use vulnerability scanners to look for known weaknesses.
(Automatic tools search for holes in the system.)

Validation
→ Check if the found vulnerabilities are real, not false alarms.
(Make sure the results are accurate.)

Remediation
→ Fix the problems or reduce the risks.
(Patch systems, update software, or change configurations.)

Reporting
→ Document what was found, what was fixed, and what still needs attention.
(Create a final report for management or security teams.)

The Loop

Steps 5 (Validation), 6 (Remediation), and 7 (Reporting) can repeat several times until everything is properly fixed.

After reporting, the process loops back to planning for the next assessment — because new vulnerabilities keep appearing.

In short:

**The Vulnerability Assessment Framework (VAF)** is a 7-step cycle that helps organizations plan, find, verify, fix, and report security weaknesses — and then repeat the process to stay secure.

Step 1: Engagement Planning (in simple words)

This is the first and one of the most important steps before starting a vulnerability assessment.

It's all about getting organized and planning everything carefully — what you'll do, how you'll do it, who's involved, and what you're allowed to test.

What happens in this step:

Planning & Logistics

Decide when and how the assessment will happen.

Make sure the right people, tools, and time are available.

Resource Management

Assign roles — who will scan, who will review, who will report.

Make sure you have enough staff, hardware, and software.

Rules of Engagement

Define what is allowed and not allowed during the assessment.

Example: Can we test production servers? Can we run scans during work hours?

Scoping

Clearly identify what systems, networks, or applications are included in the test.

This prevents scanning something you shouldn't.

Processes, Procedures, and Checklists

Create detailed steps to follow — so everyone knows the process and nothing is missed.

Access and Visibility

Make sure you have the necessary permissions and network access to perform the scans.

Why this step is crucial:

If you skip or rush planning, the whole assessment can fail.

Poor planning can lead to:

Missing important systems

Breaking something accidentally

Legal or permission issues

Wasted time and effort

That's why the saying goes:

"If you fail to plan, you plan to fail."

Example:

Imagine a company just bought another company.
Before connecting their networks, they must plan a vulnerability assessment to see what risks exist.
Without good planning — they might scan the wrong systems or cause downtime.
In short:

**Engagement Planning** means setting clear goals, permissions, scope, and resources before starting the vulnerability assessment — so the process runs smoothly, legally, and effectively.

**Step 2: Intelligence and Threat Modeling (Simple Explanation)**

After planning (Step 1), the next step is to learn about the threats your organization might face — and who could attack you and how.

This step has two main parts:

Gathering intelligence

Creating a threat model

1. **Gathering Intelligence**

This means collecting information about your organization that hackers could find and use.

Purpose:

Find out what attackers can learn about you from public or leaked sources.

Examples:

Searching the internet (OSINT) for:

Company name, website, email addresses

IP addresses and domains

Exposed data or credentials (usernames/passwords)

Technologies you use (servers, cloud providers, software)

Reading public reports, resumes, or job posts that reveal internal tools

Checking document metadata for hidden info (like author names or file paths)

All of this helps you see yourself the way an attacker sees you.

2. **Threat Modeling**

Once you know what information is out there, you create a threat model — a structured way to:

Identify who might attack you (hackers, competitors, insiders)

Understand what they might target (critical assets like databases, customer info, or email servers)

Predict how they might attack (phishing, malware, exploiting vulnerabilities)

Measure how strong your defenses are (can your security systems detect or stop these attacks?)

Why It's Important

Threat modeling helps you:

Prioritize which risks matter most

Focus your defenses on real threats

Avoid wasting time on unlikely attacks

It guides the entire security strategy — from scanning to patching.

Example:

Let’s say your company runs an online banking site.
From intelligence gathering, you find:

Public IPs of your servers

Employee email addresses on LinkedIn

Then, in threat modeling, you realize:

Hackers might try phishing emails to steal credentials

Attackers could exploit web app vulnerabilities

Your firewall can block some attacks, but not all

So, you focus your defenses on those high-risk areas first.

In short:

**Step 2: Intelligence and Threat Modeling** means collecting information about your organization and identifying which threats are most likely to attack you — so you can prepare and defend against them effectively.

**Step 3: Discovery (Simple Explanation)**

After planning (Step 1) and understanding threats (Step 2), the next step is to find out what systems and devices actually exist in the organization’s network.

You can’t protect what you don’t know you have — so discovery is about finding everything that’s connected.

Goal:

To identify all assets — computers, servers, routers, printers, websites, etc. — and see which ones are reachable and visible on the network.

Inputs:

Information you start with:

Domain names (like example.com)

Hostnames (like mail.example.com)

Subnets (network segments like 192.168.1.0/24)

CIDR ranges (to define IP ranges)

Knowledge of which networks can be reached (routable)

What kind of visibility or access you have to those systems

How It's Done:

Discovery can be done using two main types of methods:

Active Methods

Sending network probes (like pings or port scans) to see what responds.

Tools like Nmap are often used.

This helps identify live hosts, open ports, and running services.

Passive Methods

Observing network traffic without sending anything.

Helps find devices quietly, without alerting anyone.

Tools may watch logs, traffic flows, or DNS queries.

Result (Output):

After discovery, you'll have a map of the network showing:

Which systems exist

Which ones are reachable

What devices and IPs are active

Which subnets are being used

Where you have (or lack) visibility

This gives you a complete list of assets to scan in the next step.

In short:

**Discovery** is about finding all the devices and systems on your network — figuring out what's there, what's alive, and what you can reach — before you start scanning for vulnerabilities.

**Step 4: Scanning (Simple Explanation)**

After discovering all systems in Step 3, the next step is to scan those systems to find security weaknesses (vulnerabilities).

This is where automated tools are used to check for missing patches, misconfigurations, or known security flaws across the network.

Goal:

To detect vulnerabilities in:

Networks

Hosts (computers, servers)

Applications

Domains

The scan results help the organization understand where the risks are and what needs fixing.

**What to Scan:**

Network zones → Different parts of the network (like internal, DMZ, or cloud).

Hostnames → Identified devices or servers.

IP address ranges → Groups of systems found during discovery.

Applications → Web apps, databases, email servers, etc.

Domains → Public-facing systems like websites.

How Scanning Works:

Scanners send probes or requests to devices to identify open ports, services, and known weaknesses.

They compare the results against a vulnerability database (like CVE lists).

The scan should match your security roadmap — meaning it should align with what your organization wants to protect and measure.

Key Considerations:

Scanner Placement

Place scanners in the right network zones to reach the targets.

Example: one inside the internal network, one for internet-facing systems.

Testing Impediments

Some devices (like firewalls, load balancers, or intrusion prevention systems) can block scans, making results incomplete.

Scanning Strategies

Different techniques are used for different systems — like fast scans for big networks, deeper scans for critical servers.

Accuracy & Efficiency

Accuracy: Avoid false positives (mistaken results).

Efficiency: Scan quickly without slowing down or crashing systems.

Outputs (Results):

A list of found vulnerabilities

Their severity (using systems like CVSS)

Data for reporting and prioritizing fixes

Metrics to measure security progress over time

In short:

Scanning is the process of automatically checking all discovered systems for security weaknesses.
It must be done carefully and strategically to give accurate, useful results that guide the next step — fixing the problems.

**Step 5: Validation (Easy Explanation)**

After the scanning step, you now have a list of possible vulnerabilities found by the scanner.
But not all of them are real or important — some might be false alarms or low impact.

The Validation step is where you double-check those results to confirm which vulnerabilities are real, how serious they are, and how confident you are in each finding.

Goal:

To:

Confirm if each vulnerability is real.

Assign a confidence value (how sure you are that the issue exists).

Prepare validated results for the next step (remediation and reporting).

How Validation Works:

Here's what happens in this phase:

Review Each Finding

Go through every result from the scan and verify it manually or through additional tools.

Example checks:

Is that software version really vulnerable?

Is a patch truly missing?

Is there a misconfiguration in the settings?

Assign Confidence Levels

Mark each issue with a confidence rating:

✅ High confidence → Definitely real issue.

Medium confidence → Likely real but needs more proof.

❓ Low confidence → Probably a false alarm.

(Sometimes) Perform Exploitation or Testing

In rare or advanced cases, you might safely test or exploit the vulnerability (like in penetration testing) to confirm its real-world impact — but usually this isn't required in normal vulnerability assessments.

Meet Compliance Requirements

Make sure your validation results meet audit or compliance rules (e.g., PCI-DSS, ISO 27001, or internal policies).

Handle Collaboration and Data

Large organizations face challenges in sharing validation data between teams and keeping everything organized — that's why good collaboration tools and data management are important.

Inputs:

The list of potential vulnerabilities from Step 4 (Scanning).

Outputs:

A list of validated vulnerabilities (confirmed and real).

A list of unvalidated findings (not confirmed), with reasons why.

Each finding has a confidence value attached.

In short:

Validation means checking which vulnerabilities from the scan are real, how confident you are about them, and documenting the confirmed results for fixing or reporting.

Or even simpler:

It's like double-checking your homework before handing it in — making sure every vulnerability really exists and is correctly identified.

**Step 6: Remediation (Easy Explanation)**

After validating which vulnerabilities are real in Step 5, now it's time to fix them — this is the Remediation phase.

This step is where the organization takes action to eliminate or reduce the risks from the confirmed vulnerabilities.

Goal:

Fix or reduce the impact of vulnerabilities.

Prioritize changes based on risk and cost.

Integrate remediation with other enterprise systems and processes.

How It Works:

Assign Risk and Priority

Each validated vulnerability is given a risk level (e.g., high, medium, low).

Focus first on the ones that could cause the most damage or are easiest to exploit.

Leadership should review and approve the priorities.

Plan the Fixes (Remediation Options)

Decide the best way to address each issue:

Apply patches or updates (e.g., Microsoft security patch).

Reconfigure systems (e.g., close unnecessary ports, disable weak services).

Mitigate risks (use firewalls, access control, or segmentation if you can't patch).

Architectural changes (in big cases, redesign part of the system for better security).

Integrate with Enterprise Systems

Use existing company tools to track and manage fixes, such as:

✅ Change management systems (to approve updates).

Configuration management (to maintain consistency).

Ticketing systems (to assign and follow up on remediation tasks).

Framework integration (tie results into IT or security frameworks like NIST, ISO, or CIS Controls).

Perform Cost-Benefit Analysis

Not all vulnerabilities can be fixed immediately — evaluate:

Risk if not fixed vs. cost/time to fix.

Choose the most effective and realistic actions.

Implement the Changes

Execute the approved fixes and monitor to make sure they are effective.

Inputs:

The validated vulnerabilities from Step 5.

Outputs:

Fixed or mitigated vulnerabilities.

Updated system configurations and architecture.

Documentation of what was done and approved.

In short:

Remediation means taking the confirmed vulnerabilities and actually fixing or mitigating them — based on risk, cost, and business priorities.

**Step 7: Reporting (Easy Explanation)**

After fixing or mitigating the vulnerabilities in Step 6 (Remediation), the final step is to report everything — what was found, what was fixed, what remains, and what risks still exist.

This step documents the entire vulnerability management process and ensures that management and teams understand the organization's current security posture.

Purpose:

Communicate results clearly to both technical teams and executives.

Provide a complete record of vulnerabilities, actions taken, and remaining risks.

Support future improvements and compliance requirements.

How It Works:
Perform Analysis

Review all the data collected during validation and remediation.

Identify:

Which vulnerabilities were fixed.

Which ones remain open.

What risks are still present and why (e.g., cost, system dependency, business decision).

Write Findings Reports

The findings report includes:

A list of all identified vulnerabilities.

Each item's risk level (high, medium, low).

The impact if it's exploited.

Recommendations on how to fix or mitigate them.

Action plans that were taken or are still needed.

Residual risk — the remaining risk after fixes are done.

Links to policies that were violated or affected.

Executive Summary

A short, non-technical overview for leadership and management.

Explains:

The overall security health of the organization.

The most critical risks.

Progress made in remediation.

Next steps or budget needs for improving security.

It avoids technical jargon and focuses on business impact.

Each Finding Should Include:
Component Description
Policy Statement What company policy or standard was violated.
Initial Risk Statement The problem identified.
Quantitative Risk & Impact The measurable effect (e.g., data exposure, downtime risk).
Initial Risk Rating The severity before any fixes.
Mitigations Steps taken to reduce the risk.
Mitigated Risk Rating The updated severity after fixes.
Action Plans / Contingency Plans for anything not yet fixed.
Residual Risk Rating The remaining level of risk after all actions are completed.
Outputs:

Comprehensive Risk Assessment Report

Executive Summary

Detailed Findings and Analysis

Recommendations and Action Plans

In short:

**Reporting** is the final phase where you summarize all the findings, actions, and remaining risks — providing clear documentation for both technical teams and management.

**Vulnerability Criticality**

Ratings applied to identified vulnerabilities are likely subjective and have different meanings to different organizations

Rating Examples
- Low
- Medium
- High
- Severe
- Grave

It is common for the criticality determination process to be based on models such as the OWASP Risk Rating
Methodology where risk = likehood * impact
low - <100000 usd
medium - between 100000 to 49999

high - between 500000 to 99999
Severe - between 100K to 250K
Grave bigger than 250k

“Cerebration over Computation: Heartbleed” – Explained Simply

Main idea:
Don’t rely only on numbers or tools to judge how dangerous a vulnerability is. Use human thinking (cerebration) and consider real-world risk to your organization.

لا تعتمد فقط على الأرقام أو الأدوات لتقييم خطورة الثغرة الأمنية. استخدم المنطق البشري.
The Heartbleed example

Heartbleed (CVE-2014-0160) was a big internet security bug found in 2014.

It allowed attackers to steal sensitive data (like passwords and keys) from websites using OpenSSL.

The CVSS score (a number system for rating severity) gave it only 5.0/10, which looks “medium.”

But in reality, it was very dangerous and “broke the internet” because it affected millions of systems.

✅ Lesson: A “medium” score doesn’t always mean “not urgent.”

VSS and other rating systems

Tools like CVSS try to measure vulnerability risk using formulas.

Example: another bug (CVE-2012-0152) that could crash Microsoft Terminal Servers had a score of 4.3/10, which sounds low.

But if it affects your company’s critical servers, it could be a big problem!

The takeaway

Don’t just follow tool ratings blindly.

Think about your organization’s context:

How much damage could this cause here?

How easy is it to exploit?

**What systems are affected?**

Different organizations may rate the same issue differently — for example:

Microsoft may call local privilege escalation “Important,”

But your company might call it “Critical” if it exposes sensitive systems.

**Common Vulnerability Scoring System (CVSS) – Simplified**

What it is:
The CVSS is a standard system used to give a numerical score (0–10) to security vulnerabilities.
It helps organizations decide how serious a vulnerability is and how fast they should fix it.

Who created it

Developed by a working group under the U.S. Department of Homeland Security (DHS).

Includes input from companies like Cisco, Symantec, Microsoft, Qualys, eBay, and CERT/CC.

Goal: Create a common language for describing vulnerability severity.

CVSS Versions
CVSS v2 (older version)

It was useful but had some weaknesses — it did not consider:

Targets of opportunity (whether attackers are likely to exploit it easily)

Active attacks in the wild (if it’s already being used by hackers)

Ease of exploitation (how simple it is to use the vulnerability)

In short: v2 focused more on technical impact, not real-world risk.

CVSS v3 (improved version)

**CVSS v3 added environmental factors** — meaning it now looks at how the vulnerability affects your specific organization.
It includes:

Base Modifiers

Impact Metrics

Impact Subscore Modifiers

These make the score more accurate for your situation.

CVSS v3 Score Ratings

| Rating | Score Range | Meaning |
|---|---|---|
| Critical | 9.0–10.0 | Needs immediate fix — very dangerous |
| High | 7.0–8.9 | Serious, should be fixed soon |
| Medium | 4.0–6.9 | Manageable risk |
| Low | 0.1–3.9 | Minor risk |
| None | 0.0 | No real threat |

Example: Heartbleed (CVE-2014-0160)

CVSS v2 gave it a 5.0/10 (medium).

But it allowed sensitive data leaks — like passwords and encryption keys — which could lead to bigger attacks.

This shows that the CVSS score alone doesn't always reflect true danger — context matters.

**Dangers of Auto-Generated Risk Ratings — Simplified**

Main idea:
Automatic tools that calculate risk scores (like Microsoft's DREAD model) can't always judge real-world risk correctly. Human judgment is still essential.

What is an auto-generated risk rating?

It's when a computer or tool automatically gives a numerical risk score based on formulas and data — for example:

Microsoft's DREAD model (stands for:
Damage, Reproducibility, Exploitability, Affected users, Discoverability).

Each factor gets a score, and the tool combines them to calculate a total risk level (like Low, Medium, or High).

What's the problem?

**Every situation is unique (data singularity)**
No two systems, organizations, or vulnerabilities are exactly the same — but automated systems treat them as if they were.

Algorithmic limitations

Some tools are too specific, so their results are narrow and not generalizable.

Others are too broad, giving inconsistent or vague ratings.

Lack of context
The tool doesn't know your organization's setup, data value, or business priorities — so its score might not reflect true risk.

Example

An automated tool might rate a vulnerability as "Medium risk" because it's hard to exploit,
but if it affects your company's main financial system, it's actually Critical to you.

The takeaway

Automated tools like DREAD are helpful for starting the risk analysis.

But accurate risk assessment requires human review and customization for each application or environment.

That's why Microsoft stopped maintaining DREAD, though some other organizations still use it.

**Customized Risk Calculation — Simplified**

Main idea:
Don't just rely on standard scores.
Every organization should customize how it rates risks based on what truly matters to it — such as impact, importance, and likelihood.

What does a "High," "Critical," or "10" rating really mean?

These ratings usually come from formulas and weighted criteria, not from human judgment.

Each framework (like CVSS, DREAD, etc.) uses different criteria, so the same vulnerability could get different scores depending on which system you use.

Example:
One tool might give a vulnerability a 9.5, another might call it Medium, because they measure different things (like exploit difficulty vs. business impact).

Problem with “one-size-fits-all” systems

Most rating systems focus only on the technical severity —
for example, how bad the bug is from a computer’s point of view —
but that’s only part of the picture.

Some systems ignore other important things, like:

How likely the vulnerability is to be exploited in your environment

How valuable or sensitive your data is

How much financial or business damage it would cause

The Solution: Customization

Your organization should build a custom risk model that fits its needs — something flexible.

. Meltdown can allow a lower-privileged process
to read data from a higher-privileged process There are patches for some operating systems; however, they
have been problematic. Spectre can allow one process to read data from other processes at the same level of
privilege. It can be exploited using JavaScript in a browser
Meltdown is a tool

Not all Vulnerabilities Have CVSS Scores
Main idea:
Some security problems don’t get CVSS scores because they aren’t software bugs — they come from misconfigurations or abused features that admins accidentally leave enabled.

Why some vulnerabilities have no CVSS score

CVSS scores are usually assigned to known software flaws (like bugs in code).

But many real-world attacks come from bad configurations or unsafe default settings, not from bugs.

Since those are not “vulnerabilities in code,” they often don’t get CVE IDs or CVSS scores — even though they can be very dangerous.

Example 1: Cisco DTP (Dynamic Trunking Protocol)

DTP lets a switch decide if a port should be a trunk (carries multiple VLANs) or a normal port.

If this feature is left enabled, an attacker can pretend to be a switch, negotiate a trunk, and gain access to all VLANs on that switch.

This is not a bug, just a feature misused because an admin didn’t disable it.

**What Is Penetration Testing?**

Penetration testing involves modeling the techniques used by real-world computer attackers to find vulnerabilities; to exploit those flaws under controlled circumstances; in a professional, safe manner according to a carefully designed scope and Rules of Engagement; to determine business risk and potential impact, all with the goal of helping the organization improve security practices.

SEC560

Goal
Report all exploitable vulnerabilities under controlled circumstances.

Effort
~10% tools based and ~90% manual

testing
Frequency
~once per year

Customer

System owners,
operations,
engineering,
and application
stakeholders

**Red Team**

A Red Team emulates Tactics, Techniques, and Procedures (TTPs) of real adversaries to improve thepeople, processes, andtechnology inthe target environment. "The practice of looking at a problem or situation from the perspective of an adversary"– RedTeamJournal

Goal
Train and measure
blue teams'
detection and
response policies,
procedures, and
technologies are
effective.

Effort
Manual; some
Red Team
Automation tools

Frequency
Intelligence-led
(new exploit,
tool, or TTP)

Customer
Blue Teams

**Adversary Emulation — Simplified**

Main idea:
Adversary emulation is a special type of Red Team exercise where the team acts exactly like a real attacker, using the same tools, tactics, and techniques (TTPs) to test how well an organization could defend itself in a real attack.

What it means

The Red Team doesn't just look for random vulnerabilities.

Instead, they mimic a real-world threat actor — for example, a specific hacking group (like APT29 or FIN7).

They follow a structured approach (often using a cyber kill chain or MITRE ATT&CK framework) to simulate how an attack unfolds from start to finish.

Goal

Recreate a real, end-to-end cyberattack scenario.

Measure how well the organization:

Detects the attack,

Responds to it,

And protects its critical assets.

The result gives a holistic (overall) picture of the company's readiness for sophisticated, real-world attacks.

Effort

Mostly manual — Red Teamers carefully plan and execute realistic steps.

More setup and planning than a normal penetration test (since they're emulating full adversary behavior, not just scanning for bugs).

Frequency

Usually done once or twice a year, because it takes a lot of time and coordination.

Customer / Scope

The entire organization — not just one system or app.

Tests people, processes, and technology together (e.g., how well employees, SOC analysts, and systems react to an attack).

**Purple Team — Simplified**

Main idea:
A Purple Team is not a separate team — it's a collaboration process where the Red Team (attackers) and Blue Team (defenders) work together to improve the organization's security.

What it means

Normally, Red Teams attack without telling the Blue Team (to test real responses).

But in Purple Teaming, both sides cooperate openly.

The Red Team shows the Blue Team exactly how attacks happen, step by step.

The Blue Team uses this knowledge to improve detection, response, and defense in real time.

Think of it like a live training session between attackers and defenders.

Goal

Red Team emulates real attacker behavior (TTPs).

Blue Team observes and learns, improving:

Security monitoring,

Alert tuning,

Response procedures,

And defensive technologies.

✅ The main goal is immediate improvement, not secrecy or "catching" the Red Team.

Effort

Manual work, requiring coordination between both teams.

Exercises are interactive and focus on learning, not competition.

Frequency

Done as needed, often intelligence-led — for example:

When a new exploit, hacking tool, or attack technique (TTP) appears.

Or after a Red Team exercise, to fix the weaknesses found.

**Rules of Engagement (RoE) — Simplified**

Main idea:
Before any penetration test or Red Team exercise begins, everyone must agree on clear rules about how the test will be done.
This written agreement is called the Rules of Engagement (RoE) — it ensures safety, legality, and fairness.

What it is

The Rules of Engagement define:

What testers can and cannot do,

When and how the testing will happen,

And who to contact if something goes wrong.

It protects both the organization being tested and the testers.

Key Elements of Rules of Engagement
Rule Area Description
Start & End Dates Clearly state when the testing begins and ends.
Permitted Time of Day Decide if testing can happen during work hours or only at night/off-hours to avoid business disruption.
Contact Information Include names and phone numbers for testers, system owners, and business contacts in case of emergency.
Data Restrictions Define what data can or cannot be accessed — e.g., PII (Personally Identifiable Information), financial data, or sensitive customer info.
Response to Blocking Explain what to do if firewalls or intrusion systems block the testers' IPs or if new rules are added mid-test.
Reporting Method Specify how and in what format the final results or findings must be submitted.
Why it's important

Prevents misunderstandings between testers and clients.

Ensures testing stays within legal and ethical boundaries.

Protects the organization's systems and data from accidental damage.

Required for insurance and liability protection — if testers go outside the agreed scope, it could cause legal trouble or financial penalties.

Simple takeaway

The Rules of Engagement are like the “contract and safety plan” for a penetration test — they make sure everyone knows the limits, responsibilities, and procedures before any hacking begins.

**Scoping — Simplified**

Main idea:
The scope tells the penetration testers exactly what they may and may not attack. A clear scope protects the customer, keeps testing legal/safe, and determines how realistic the test can be.

What scoping defines

Which systems, networks, domains, and applications are in-scope (allowed targets).

Which systems are out-of-scope (must not be touched — often high-value or fragile systems).

Rules for cloud or vendor-hosted systems (these often need extra approvals).

What to do if a new/unidentified system is discovered during testing.

Important rules for discovered systems

If a tester finds a system that’s not listed as in-scope or out-of-scope, assume it’s out-of-scope.

Contact the customer: if they agree to add it, get that change in writing before testing it.

Why scope matters

Too narrow a scope → test becomes unrealistic and less valuable (real attackers don’t follow scope limits).

Testers often need to educate the customer about why expanding scope or including certain techniques increases real-world value.

Sometimes customers intentionally limit scope (e.g., to “pass an audit”) — that should be documented so expectations are clear.

Vendor / Cloud considerations

Systems in the cloud or at a third-party vendor may have policies or contracts preventing testing.

These require research and explicit approval before being included.

✅ Quick Scoping Checklist (use before testing)

List all in-scope hosts, IP ranges, applications, and domains.

List all explicitly out-of-scope/critical systems.

Define time windows and permitted testing times.

Specify handling of PII and sensitive data.

Describe the process for newly discovered systems (contact + written approval).

Confirm vendor/cloud system approvals (if applicable).

Record contact info for emergency escalation.

Get written sign-off from the customer on final scope.

TL;DR

Scoping = the rulebook for tests. Be explicit, assume unknown systems are out-of-scope until approved in writing, and remember that overly narrow scope reduces the test’s business value. Testers should help customers understand trade-offs.

Types of Penetration Testing: External
An external penetration test focuses on the perspective of an attacker from outside the organization

- Targets externally-facing systems such as DNS, Mail, and Web servers
- May include phishing attacks as another means to gain access

Types of Penetration Testing: Internal
An internal penetration test focuses on what an attacker could do if starting from within the organization's network

• This can be a malicious insider with privileged access, such as an employee, contractor, or vendor
• It could also be an attacker who has successfully gained a foothold inside the organization – We can assume a breached state

The goal is to identify what an attacker could do if on the internal network
Command &
Control and data
exfiltration

Lateral
movement

Privilege
escalation

Machine in the
Middle

Types of Penetration Testing: Web Application
Another common penetration test focuses specifically on web applications and databases
• These systems are the ones most often exposed to the Internet and provide access to sensitive data
• If poorly coded they can be exploited to gain unauthorized access

The Open Web Application Security Project (OWASP) is a fantastic resource that offers tools and guidance on web app security

nput- based attacks (Injection)
Test user inputs to see if they can inject SQL, OS commands, or other code that the server will run. (e.g., SQL injection)

Authentication & session tests
Check login flows, password resets, session cookies, multi- factor auth — can attackers impersonate users or hijack sessions?

Access control / authorization
Ensure users can't access or modify other users' data or admin functions by changing IDs, URLs, or parameters.

**Cross- Site Scripting (XSS)**

See if an attacker can inject malicious JavaScript that runs in other users' browsers.

Business‑ logic testing
Look for flaws in how the application implements real‑ world rules (pricing, checkout, workflow) that let attackers get unintended benefits.

Sensitive data exposure
Check storage/transit of secrets (passwords, credit cards). Are data encrypted? Are backups or logs leaking info?

File upload & remote code execution
Test whether uploaded files can become executable or allow attackers to run code on the server.

API & backend testing
Test REST/GraphQL endpoints, rate limits, and how the API enforces permissions.

Configuration & deployment issues
Look for unnecessary services, default credentials, exposed admin panels, or overly permissive CORS settings.

Third‑ party components / dependency vulnerabilities
Check libraries and plugins used by the app — they can contain known vulnerabilities.

Types of Penetration Testing: Social Engineering
Use of influence and persuasion to deceive people for the purpose of obtaining sensitive target information or for the victim to perform an action

Enables access to the target, usually by a sense of trust or authority of the attacker

Preys on human behavior and works to befriend the victim to obtain the information

Attempts to trick or manipulate data or access from a person

Bypasses
network security
by exploiting
human
vulnerabilities

Types of Social Engineering: An Easy Overview

Social engineering is when attackers manipulate people into giving up confidential information or performing actions they normally wouldn't. It exploits human psychology rather than relying on technical vulnerabilities. There are two main types: computer-based and human-based attacks.

1. Computer-based Social Engineering

This type of social engineering relies on technology or digital tools to deceive users into giving away sensitive information. Common methods include:

Pop-up windows
An attacker can make a pop-up appear on your computer that looks like a legitimate system warning or login prompt. For example, it might ask you to re-enter your password, claiming it's expired or needs to be verified. Most users, not thinking twice, might comply, giving the attacker their login credentials.

Mail attachments
Attackers often send fake emails that seem legitimate, with attachments or links that, when opened, install malware on the victim's device. The email could look like it’s from a trusted company, like a bank or your workplace, asking you to open the attachment for important information.

2. Human-based Social Engineering

This type of attack involves direct interaction with people to manipulate them into giving away information or granting access. These are the more traditional, non-digital tricks that rely on human psychology.

Urgency
Attackers create a sense of urgency to push the victim into acting quickly without thinking. For example, an attacker might call pretending to be from IT, saying there's been a security breach, and they need you to change your password immediately to avoid account loss. The pressure makes people more likely to fall for the scam.

Third-person authorization
This is when the attacker pretends they have authority from a third person (like a boss or department head) to request sensitive information or actions. For instance, a

scammer could call a help desk and pretend to be a high-level executive, like the CEO, who urgently needs their password reset because they've forgotten it while traveling.

Example Scenario:

Imagine an attacker calls your company's help desk, pretending to be the VP of your company. They say, "I'm traveling and I've forgotten my password. I need access to the system immediately for an important meeting. Can you help me reset it?" The help desk agent, feeling pressured, might not verify the caller's identity and simply resets the password.

Here, the attacker used impersonation (posing as a VP) and urgency (the need to act fast because of the meeting) to manipulate the help desk into giving them access.

How Social Engineering Works

These attacks take advantage of basic human traits like:

Curiosity (e.g., phishing emails with tempting offers or news)

Trust (people trusting emails or pop-ups that look official)

Fear of missing out (acting out of urgency)

Lack of knowledge or awareness (not questioning a "normal" request)

Real-World Impact

Social engineering works because people are often the weakest link in security systems. No matter how strong your firewall or encryption is, if an attacker can convince someone to open the door for them, the system is vulnerable.

How to Defend Against Social Engineering

Awareness and Training

Regularly train employees on social engineering techniques.

Teach them to question and verify requests, especially those that involve personal information or urgent actions.

Multi-factor Authentication (MFA)
Even if a user is tricked into giving away their password, MFA adds an extra layer of protection.

Email & Web Filters

Use strong email filtering to block phishing attempts, and warn users against clicking on suspicious links or downloading attachments.

Verify Requests
Always verify requests for sensitive information through a separate communication channel (e.g., call the person who supposedly made the request to confirm).

Additional Types of Penetration Testing
Mobile Device Testing
• The attack surface of mobile devices certainly includes similar attacks as those related to web clients and apps
• Mobile devices are used for much more than just the web and req

Product Security Testi ng
• This type of testing could include Internet of Things (IoT) devices, network appliances, such as VOIP phones, printers, switches, and routers, and much more
• Often involves reverse engineering, fuzzing, and debugging

Physical Penetration Testi ng
• This type of testing factors in devices such as badge readers, elevators, security gates, biometric devices, safes

Staying Focused in a Penetration Test: The Bigger Picture

Penetration testing (pentesting) is about more than just hacking your way into systems or breaking through defenses. While technical wins like gaining Domain Admin access or bypassing security controls are impressive, they're not the end goal.

The true value lies in demonstrating the real-world impact of vulnerabilities and giving actionable recommendations for remediation.

Key Points to Stay Focused On:

Technical Wins ≠ Full Value
Sure, gaining Domain Admin access is a big deal, but it's only part of the picture. The bigger question is: What can you do with that access?

Can you steal sensitive data?

Can you escalate privileges to other systems or services?

Can you disrupt operations, steal credentials, or use the access to further compromise the network?

Impact is What Matters
Gaining access to a high-level system doesn't mean much unless you can explain the potential damage or risks to the business.

For example, Domain Admin access might allow you to read sensitive financial data or delete critical system backups.

What if this level of access gives you the ability to access customer data, intellectual property, or the company's entire email system? That's the real risk.

Focus on Business Impact, Not Just Technical Success
When presenting your findings, avoid focusing only on the "how" of the attack. Ask yourself:

What does this mean for the company?

What are the financial, operational, or reputational risks?

Is there a potential for a data breach, a regulatory penalty, or brand damage?

Provide Practical Recommendations for Remediation
It's not enough to identify vulnerabilities — you need to provide clear, actionable steps for fixing them.

For example, instead of just saying "the server is vulnerable to privilege escalation," say why it's a threat and how to fix it: "The server allows users to gain admin access through a misconfigured user role. Fix: Review user permissions and implement the principle of least privilege."

Focus on long-term solutions like improving security policies, better access control, regular patching, and employee awareness training.

Real-world Scenario Example:

Technical Win:
You break into a company's network and gain Domain Admin access. You've bypassed firewalls, infiltrated the server, and elevated your privileges.

What Does This Mean?
Now, think about the consequences of that access:

Could you access sensitive customer data?

Could you erase company records or backups?

Could you deploy ransomware or malware on other systems?

Impact:
The company could lose intellectual property, face compliance violations (e.g., GDPR, HIPAA), or experience business downtime. You're not just breaking in; you're showing how it could bring down the business.

Remediation:
Now, provide practical fixes:

Patch the vulnerability: Ensure all security patches are up to date and prioritize critical vulnerabilities.

Improve access controls: Enforce the principle of least privilege for all accounts and audit admin access regularly.

Implement multi-factor authentication (MFA): Especially for admin accounts, to prevent unauthorized access.

Review monitoring tools: Set up real-time alerts for any suspicious behavior or privilege escalation attempts.

Shift in Focus: From "How" to "What"

Penetration testers need to go beyond just showing how they gained access and focus on what that access means. Here are the two questions that should always be in the back of your mind:

"What were you able to do with that level of access?"

Did you access critical data?

Could you disable security systems?

What’s the worst-case scenario for the business?

"What are your recommendations for remediation?"

How can the company fix the vulnerability?

How can they prevent similar attacks in the future?

Why It Matters:

The ultimate goal of a penetration test is to help the customer reduce risk. The focus should be on preventing real harm rather than showing off technical exploits. It’s about protecting business continuity, customer trust, and confidentiality.

Quick Summary:

While gaining access to critical systems (like Domain Admin) is a huge technical achievement, it’s the business impact and remediation that matter most. A penetration tester’s job isn’t just about breaking in — it’s about showing why it matters and guiding the company to fix those weaknesses to prevent real-world damage.

Penetration tests usually follow a helpful sequence of five phases. You can jump between them, but this order keeps work organized and focused.

**OSINT / Reconnaissance** — Gather public info about the target (domains, IPs, employees, tech stack). Goal: map the attack surface and find entry points.

**Scanning & Enumeration** — Actively probe systems to discover hosts, open ports, services, and detailed identifiers (usernames, shares). Goal: build a precise inventory of targets.

**Vulnerability Identification** — Match discovered services and configs to known weaknesses (misconfigurations, unpatched software, weak credentials). Goal: prioritize likely attack paths.

**Exploitation** — Attempt to exploit the prioritized vulnerabilities to gain access (shells, credentials, session takeover). Goal: demonstrate that a vulnerability is actually exploitable.

**Post- Exploitation** — Explore what an attacker can do with that access (data access, lateral movement, persistence) and measure business impact. Goal: show real-world consequences and collect evidence for remediation.

**Easy explanation — Open‑ Source Intelligence (OSINT) Gathering**

OSINT is the practice of collecting information about a person, company, or target from publicly available sources. “Open‑ source” here means anything you can legally access without hacking — web pages, social media, conference slides, printed reports, public records, and more.

What it looks like (simple)

You search the internet, social networks, and public records to build a picture of the target.

It’s usually passive — you try not to touch the target’s systems so you remain undetected.

The aim is to discover useful facts: domains, employee names, tech stack, public IPs, leaked credentials, published presentations, etc.

Where to look

Search engines (Google, Bing) — great starting point, but not everything is indexed.

Social media (LinkedIn, Twitter, Facebook) — people share roles, emails, projects.

Company sites & press releases — org charts, tech partners, blog posts.

Public records & filings — ownership, addresses, legal documents.

Specialized sources — Shodan for internet‑ connected devices, paste sites for leaked data, and dark‑ web forums for exposed credentials.

Conferences / slide decks / PDFs — often reveal tooling, architecture, or internal emails.

Handy tools (examples)

Shodan — find exposed devices and services on the internet.

Maltego — map relationships between people, domains, and infrastructure.

Google dorking — clever search queries to find hidden files or pages.

OSINT frameworks and browser extensions that automate collection.

Why it matters

OSINT helps you map the attack surface before any active testing. It reveals weak points (exposed services, leaked credentials, or overshared employee info) that an attacker might exploit.

Quick tips & ethics

Always follow legal and engagement rules — OSINT is public info only, no unauthorized access.

Be mindful of privacy and company policies.

Combine passive OSINT with other testing steps to prioritize real risks.

**Scanning and Enumeration**

Scanning was covered in the Vulnerability Assessment module
We will cover tools such as Nmap coming up soon!
We can enumerate discovered systems to learn information such as what services are tied to listening ports
It is not uncommon for administrators to bind services to non-standard ports, such as running an HTTPS service over TCP port 4433
Scanning and enumeration tools can help us with the aforementione

**Easy Explanation — Vulnerability Identification**

Vulnerability Identification is the phase where we find weaknesses in a system that could be exploited. It's like scanning a target to look for doors that might be unlocked (vulnerabilities) so we know where we can later "break in" (exploit).

How it works:

What we do: We identify running services (like web servers, databases, etc.) on the target system and then check if these services have any known security flaws (vulnerabilities).

Goal: Find which services or apps are vulnerable and can be used to launch attacks during the next phase (exploitation).

Example:

If a service is running, we look for its version number (e.g., Apache version 2.4.7) and check if there are any published vulnerabilities for that version.

If we find a known weakness, we move to the next phase to exploit it (try to break into the system using that flaw).

Tools:

Nessus and OpenVAS are tools that help scan for these vulnerabilities. They do this by checking the version of software and matching it to a database of known flaws.

Key difference from vulnerability assessments:

Vulnerability assessments just find the flaws but don’t try to exploit them. They’re for detection only.

In penetration testing, we not only find the vulnerabilities, but also try to exploit them to see if we can actually break into the system.

**Easy Explanation — Exploitation**

Exploitation is the phase where we take all the information we gathered (from OSINT, scanning, and identifying vulnerabilities) and use it to break into the system.

What happens in this phase:

Goal: Gain access to a system, network, or other resources (like databases or files).

We pick the most effective attack techniques based on what we know will work, and we aim to do it without getting caught.

Important Points:

**Techniques**: Exploitation could involve attacking passwords, using social engineering (tricking people), or leveraging technical vulnerabilities in the system (like flaws in software).

**AV Evasion**: Many systems have antivirus (AV) and antimalware protections that can detect when we're trying to exploit a system. To avoid detection, penetration testers and attackers often use tricks, like:

**AV-evasion techniques** (e.g., ghostwriting) that hide the malicious payload from signature-based antivirus systems.

Ghostwriting involves modifying the malicious code (like malware) at a low level, changing its signature so it looks different to AV software.

Example of Ghostwriting:

Imagine a piece of malware is designed to spread or exploit a system, but it's detected by antivirus software because of its signature. With ghostwriting, an attacker could slightly alter the code, inserting harmless instructions or moving things around so it looks different to the AV and sneaks past undetected.

**Easy explanation — Post‑ Exploitation**

Post‑ exploitation is what you do after you've already gotten a foothold in a system. The goal isn't just "I'm in" — it's to show what an attacker could actually do with that access, so the customer understands the real business impact.

Main activities (simple)

Pivoting & lateral movement
Use the compromised machine as a jumping‑ off point to reach other systems inside the network (think: moving from a single infected laptop to the internal file server).

Privilege escalation
If you're stuck as a low‑ privilege user, try to get higher rights (Administrator, Root) so you can do more damage or access more data.

**Data exfiltration**
Demonstrate the ability to read and remove sensitive information (customer records, source code, credentials) — usually done carefully or simulated during a legal test because real theft may be restricted by policy or law.

**Maintaining access (persistence)**
Put measures in place so access survives reboots or brief cleanups (e.g., creating ways to get back in), again only within the rules of an engagement.

Tools (high level)

Pen testers use frameworks (Metasploit, Empire, Covenant, Cobalt Strike, etc.) to help with these tasks — some are commercial, some open‑ source. These tools speed up legitimate testing but are the same kinds of tools attackers might use.

Why this phase matters

Initial access proves a vulnerability exists. Post‑ exploitation shows impact: could attackers read private data, shut down operations, impersonate users, or persist undetected? That's the information organizations need to prioritize fixes.

**Safety & ethics (important)**

During a real pen test, testers follow strict rules:

Work only within the defined scope and legal agreement.

Avoid destructive actions or real theft unless explicitly authorized.

Collect evidence and provide remediation guidance rather than cause business harm.

Quick defenses to stop post‑ exploitation

Network segmentation and least privilege (limit what a compromised host can reach).

Strong monitoring and EDR (detect unusual lateral movement or privilege changes).

Regular patching, MFA, and credential hygiene.

Backup and recovery plans to reduce damage from data theft or ransomware.

**Easy explanation — Port Scanning & Nmap**

Port scanning is how you check a host to see which network ports (0–65,535) are open and listening for connections. An open port usually means a service (web server, SSH, FTP, etc.) is running there — and attackers sometimes use open ports as backdoors.

The basics (very simple)

Scan all ports 0–65,535 to find services.

Do it twice: once for TCP ports and once for UDP ports (they behave differently).

A tool tries to connect to each port; if the connection succeeds, that port is listening (open).

**What Nmap is**

Nmap = free, widely used network scanner.

It finds hosts, open ports, and often which application and version are behind a port.

Supports many scan types (SYN, FIN, ACK, UDP, ping sweeps) and extras like remote OS detection and service/version detection.

**TCP vs UDP (short)**

TCP is connection- oriented. Scans often rely on connection setup (SYN, ACK).

UDP is connectionless and trickier to scan (responses are different), so UDP scans can be slower and noisier.

Common Nmap features (what it can do)

Find which hosts are up (network mapping).

Detect open TCP/UDP ports and the services behind them.

Fingerprint remote OS and software versions.

Run different scan techniques (stealth scans, full connect scans).

Scriptable checks for known issues (Nmap Scripting Engine).

**Easy explanation — Port Scanning with Nmap**

Nmap is a powerful network scanner that tells you which hosts are up, which ports are listening, and often which services (and versions) are running. Here's a simple, friendly breakdown of the demo output and the main concepts:

What the example command does

nmap -A -T4 10.10.10.10

-A = enable aggressive detection (service/version detection, OS detection, scripts).

-T4 = faster timing (quicker scan, slightly noisier).

10.10.10.10 = the target IP.

The output shows:

A short list of interesting ports found (others are filtered).

For each port: PORT STATE SERVICE VERSION (e.g., 22/tcp open ssh OpenSSH 4.3).

Device type / OS guess (e.g., Linux 2.6.x, Fedora).

Extra info like the HTML title for a web server.

Port states (what they mean)

open — something is listening on that port (a service is active).

closed — the port is reachable but nothing is listening there.

filtered — Nmap couldn't tell (often because a firewall blocked the probe).
Knowing a port is open only proves “something responded,” not exactly what it is.

Service detection caveat

Port numbers are only hints. Just because port 80 returned a response, you might expect HTTP — but attackers or admins can run other services on port 80 to bypass filters or hide.

Nmap goes further by analyzing responses (fingerprinting) to guess the actual service and version. These fingerprints improve over time as users submit matches to the Nmap database.

Why scan the whole network

Scanning a single host is useful, but to secure a network you often need to scan entire ranges (wildcards, ranges, CIDR) to discover rogue devices (unauthorized routers, APs, services).

Nmap supports many ways to specify multiple targets: lists, wildcards (e.g., 192.168.1.*), ranges (192.168.1.10-35), and CIDR (192.168.1.0/24).

Useful Nmap tips / features

Scans for both TCP and UDP (UDP is slower and noisier).

Supports many scan types (SYN, FIN, ACK, ping sweeps, stealth scans).

Can detect OS and application version info (not 100% guaranteed).

-randomize_hosts scans hosts in random order to be less predictable.

Be cautious: scanning can trigger alarms—always have authorization.

Quick common commands

Basic scan: nmap target.com

Scan all TCP ports: nmap -p- target.com

Service & OS detection: nmap -A target.com

UDP scan: nmap -sU target.com

Operating System Identification

Operating System Identification

Maybe the most interesting of Nmap's features is its ability to probe a target and determine what operating system and version it runs. Think for a second how important this can be to attackers. If they know a particular machine is running Windows 10, for example, they can save themselves a lot of time by trying only Windows oriented attacks and skipping all the UNIX and Linux exploits, plus those specific to other versions of Windows.

Metasploit
-Exploitation framework
-If a module for a vulnerability exists, Metasploit will attempt to exploit the target and provide access

Metasploit

Vulnerability scanners are useful for finding vulnerabilities, but because they are often looking for conditions of a vulnerability, they can have false positives. The next evolution in tools is exploitation frameworks. These are tools that, if a vulnerability exists, will exploit the system and provide the person running the tool access to the system. One very popular tool in this space is Metasploit. Metasploit will exploit a vulnerability if it is present and provide access to the system through various methods that can be specified in the tool.

Meterpreter

What it does (simple):

After the exploit works, Meterpreter runs on the victim machine and opens a secure (TLS) connection back to the attacker — like a locked, private chat channel.

Instead of a normal command prompt, the attacker gets a feature-rich interactive menu (a “metered” shell) that can do many advanced things without leaving obvious traces on disk.

Why attackers like it:

In-memory execution: it often runs only in RAM (not as files on disk), making it stealthier.

Encrypted connection: uses TLS so traffic is harder to inspect.

Modular: the attacker can load extra capabilities on demand (file access, screenshots, webcam/mic access, password harvesting, process spawning, network sniffing, etc.).

Powerful post-exploitation tools: impersonate users, pivot to other systems, capture keystrokes, and more.

How it’s different from a normal remote shell:

A normal shell is basic (run commands, see output). Meterpreter is an advanced toolkit — it’s designed for covert, flexible control and extension while minimizing footprints on the target system.

Quick examples of capabilities:

Browse and download/upload files from the victim.

Take screenshots or control the webcam/mic.

Dump passwords or steal tokens.

Run network sniffing on the victim’s interfaces.

Spawn new processes or inject into existing ones.

Move laterally to other machines.

Defensive notes (how to reduce risk):

Keep systems patched and reduce attack surface.

Use EDR/antivirus that detects in-memory payloads and suspicious behaviors.

Limit user privileges (no admin by default).

Monitor outbound encrypted connections and unusual long-lived TLS sessions.

**Network segmentation and least privilege for services.**

What they are (simple)

Think of a C2 (Command & Control) framework as a headquarters and implant as the secret agent installed on a compromised computer.

The implant is a small program that runs on a victim machine and waits for orders.

The C2 framework is the control panel the operator uses to talk to implants, send commands, collect data, and coordinate movement across a network.

What they do (everyday words)

Covert communications: implants regularly check in with the C2 to get tasks and send back results, often disguised or encrypted.

Post-exploitation actions: run commands, steal files, capture credentials, take screenshots, record audio, etc.

Lateral movement / pivoting: use one compromised host as a stepping-stone to reach others.

Automation & orchestration: operators can run many implants at once, schedule actions, and reuse scripts/modules.

Popular examples (very short)

Empire — focused on PowerShell (and Python originally). Great for PowerShell-based automation; original project reduced due to abuse.

Cobalt Strike — commercial adversary-emulation product used by red teams (and abused by attackers); supports phishing, covert channels, pivoting, and lots of post-ex stuff.

Covenant — open-source .NET C2 framework similar goals to Empire but using .NET tooling.

Why red teams use them (vs. one-off scripts)

Reusable: frameworks provide many ready-made tools so teams don’t reinvent the wheel.

Realistic: they simulate advanced attacker tradecraft (persistence, stealthy comms, lateral movement).

Automates complex operations: handle many targets, schedule tasks, load modules dynamically.

Why defenders fear them

Stealthy: implants often use encrypted/obfuscated traffic and live in memory to avoid detection.

Powerful: they can escalate privileges, move laterally, and steal sensitive data.

Modular: operators can add new capabilities on the fly, making detection harder.

The C2 Matrix

Google doc of most C2 frameworks

- Documents various capabilities of each framework
- There is no right or wrong, better or worse framework

the document all public command and control frameworks in a single place AS a red team objective is different the command-control framework may be need to be different as well

Password Compromise

Password cracking techniques were covered in Book 2; however, we want to continue looking at the topic through the eyes of a penetration tester. Password compromise can come in many forms, from cracking and guessing to reuse and spraying. We will take a look at a couple of these on the upcoming slides. The important thing to know, from a defensive perspective, is that proper use of Multi-Factor Authentication (MFA) using solutions based on the FIDO2 standard, such as YubiKeys, can greatly improve your security

Password Reuse and Stuffing

Password reuse is the common practice of using the same credentials for many resources. Some users are diligent about setting up unique credentials for each website where they authenticate, but many simply reuse the same username and password combinations. Password wallets are a popular way of trying to mitigate the

likelihood of reuse. Regardless, if someone can determine your credentials for one website or resource, there is
a good chance they will have success using these same credentials at another resource

Password Spraying
Password spraying is a simple concept where commonly used
passwords are tried against many user accounts
• Helps prevent against account lockout as one
password is tried across thousands of accounts
• By the time a second password is tried, many sites
will have reset the account lockout security
threshold
• Typically targets web and cloud sites
Password Guess: $ummer2021

**Responder — simple explanation**

Think of Responder like a mischievous helper that listens on a local network and pretends to be services machines ask for when DNS fails. It's a tool pen-testers use to capture credentials when devices try to look up names and can't reach a real DNS server.

What it does (plain words)

When a computer can't resolve a name (DNS fails), it may broadcast a request on the local network (LLMNR, mDNS, or NetBIOS/ NBT-NS).

Responder answers those broadcasts faster than the real host and claims "I'm the server you asked for."

The requesting machine then tries to authenticate to Responder and sends back credential material (often hashed Windows credentials like NetNTLM hashes).

The tester can then try to crack those hashes offline or use them in relay attacks to access other resources.

Why attackers and testers use it

Easy way to harvest credentials on a flat or poorly segmented network.

Works especially well when name resolution falls back to broadcast protocols (LLMNR/mDNS/NBT-NS) — a common misconfiguration.

Great for demonstrating risk during red-team/pen-test engagements.

Marriott Data Breach
What Happened

Hackers broke into Marriott's hotel network (specifically the Starwood guest database).

They stayed undetected for 4 years — from 2014 to 2018!

They stole between 380–500 million customer records — including names, addresses, passport numbers, and other private info.

It was a confidentiality attack — meaning secret data was taken, not changed or destroyed.

Why It's Important

It shows that even big companies can miss attacks for years if they don't monitor properly.

The attack could have been prevented or detected earlier with basic security checks and good logging.

There were red flags and logs, but no one acted on them.

It's a lesson that "Prevention is ideal, but detection is a must."
→ You can't stop every hacker, but you must notice when something strange is happening.

What Went Wrong

Weak or missing monitoring of network activity.

Possible outdated systems with unpatched vulnerabilities.

Poor incident detection and response.

No one checking unusual access patterns for years.

✅ What Could Have Helped

Regular security monitoring and log review.

Stronger access control (limit who can see sensitive data).

Frequent vulnerability scans and updates.

A solid incident response plan to react fast when signs appear.

Data encryption to protect customer info even if stolen.

The Big Lesson

You might not stop every hacker, but if you watch your systems closely, you can catch them early and reduce the damage.

Breach Summary
What Happened

Breach discovered: September 8, 2018

Publicly reported: November 30, 2018 (almost 3 months later)

Attack started: back in 2014 — and wasn't noticed for 4 years!

Data stolen:

9.1 million payment card details

24 million passport numbers

Around 383–500 million total customer records

How It Happened

Marriott bought Starwood Hotels in 2016.

But the old Starwood systems were never fully merged into Marriott's main network.

Hackers were already hiding inside Starwood's IT systems before and after the purchase.

Because those systems stayed separate and outdated, attackers could keep stealing data quietly.

Why It's a Big Deal

Marriott waited months to tell the public — like a doctor finding a serious problem and not telling you right away.

The breach showed how poor system integration and weak monitoring can cause huge damage.

The stolen data included highly personal information — passports, addresses, and payment details.

Marriott was fined £99 million (about $120 million) by the UK's ICO for poor data protection.

The Big Lesson

When companies merge or buy others, they must secure and monitor inherited systems immediately.
Also, detecting and reporting breaches quickly is vital to protect customers and maintain trust.

Breach Impact (1)

What the Hackers Got

After the Marriott systems were hacked, attackers accessed sensitive customer data, including:

Reservation database

Credit card numbers

Passport numbers

Mailing addresses

Names, phone numbers, and emails

Why It's Serious

This wasn't just a small leak — hackers got personal and financial data that could be used for identity theft or fraud.

The stolen data could be sold on the dark web or used to open fake accounts, make purchases, or steal money.

For customers, that means money loss, stress, and years of repairing credit or identity issues.

Business Damage

Marriott's reputation was badly hurt — customers lost trust.

The company faced massive financial costs: fines, lawsuits, and recovery expenses.

This kind of breach shows that not only prevention matters — continuous monitoring is critical.
→ If they had noticed large data transfers or strange activity earlier, the attack might have been stopped sooner.

Key Lesson

It's not just how attackers break in — it's what they can access once they're inside. Always watch your systems for signs of data theft, just like doctors check your vital signs for early warning.

Too Little, Too Soon (1)
What Marriott Did After the Breach

After the data breach, Marriott tried to fix things by:

Giving customers free credit monitoring (Web Watcher) to help them spot identity theft.

Setting up a website with information about the incident.

Spending millions on new security tools and systems.

Dedicating thousands of work hours to investigate and find the root cause.

What Went Wrong

The response website they made had no HTTPS at first (so it wasn't fully secure).

They sent emails about it that looked fake or suspicious, similar to phishing messages.

This caused confusion and risk — attackers could have used this mistake to launch phishing scams.

It showed that even after a major breach, poor planning and rushed actions can make things worse.

The Big Lesson

Fixing a breach after it happens is much more expensive than preventing it in the first place.

Security should be planned early, not added later as a reaction.

Even during incident response, organizations must stay careful and follow good security practices.

Key Takeaway

Marriott’s quick but sloppy response showed that security must come first — not after the damage is done.

Fallout: Too Little, Too Soon (2)

Imagine you have just found out that your entire internet profile including all your personal information has been breached. You go to Marriott's website to see what is going on and you are pointed to a website that can answer all your questions. You get to that website, and it is poorly designed and executed. Indeed, the breach website URL was info.starwoodhotels.com, which redirected to https://answers.kroll.com/. This site was not on the corporate domain, which made it look like a phishing site. Marriott also did not give the ability to check if you were part of the breach but had opted to send emails on a rolling basis as new email addresses turn up. Eventually, this was adjusted: The page was moved to the Starwoods domain, and a series of hotlines were set up.

**WannaCry Ransomware (1)**

Few have not heard about the worldwide cyber attack called WannaCry. Remember that doctor example from the Marriott breach? Let's change the scenario a bit. Imagine all your personal medical history is stored on a server the hospital owns. What if all your information was encrypted and made unavailable to both you and your doctor? Now what if you were in critical condition and were about to go into surgery? Many modern hospitals rely on electronic records for patients. If that patient information is not available when it is needed due to ransomware, it could literally be a life-or-death situatio

**Breach Impact (2)**

Believe it or not, ransomware is a business that runs like many others. Attackers go to work and attempt to meet a standard for their performance just like the rest of us do. For ransomware, the business model is this: Encrypt the data, charge a fee that seems reasonable for the service of decryption, and return the data without harm. These adversaries know that if they follow the business model, then they will stay in business. People would rather pay a small "maintenance" fee than permanently lose their data.

Root Cause

Forensic investigations found that the attack began on May 12, 2017, in Asia. What's notable about this attack is that it was not a virus and was not the payload of a specific, targeted attack. It is considered a network worm because it also includes a "transport" mechanism to automatically spread itself. WannaCry infects a system and then probes the internal network for similarly vulnerable systems. Because the attack exploited a vulnerability in Microsoft's SMB protocol, the most widely impacted operating systems were Windows systems. Similar to the Apache Struts-2 vulnerability, a patch had already been released for nearly two months before the WannaCry attacks began

Virus: Needs a host program, spreads via user interaction, can corrupt files or slow down systems.

Worm: Self-replicating, spreads automatically across networks, can overload systems or networks, and doesn't require user action.

WannaCry Tools

The victims of WannaCry were not targeted and were not "special" to the attackers. They were just organizations that were not following security fundamentals such as patching and account management. The WannaCry worm took advantage of these gaps in security. The attack had multiple layers and utilized two key tools for exploitation: Eternal Blue and DoublePulsar.

Eternal Blue exploits Microsoft's Server Message Block protocol version 1. WannaCry allowed for remote
code execution on victim systems. SMBv1 mishandles specially crafted packets allowing arbitrary code
execution on the target system. DoublePulsar is a trojan backdoor tool. Once successfully executed, it runs in
kernel mode, allowing for privileged access on compromised systems

WannaCry uses three primary commands
Ping: diagnostic tool used to test system
reachability
Kill: system command used to terminate running
processes
Exec: which can be used to load and execute
malware on the victim system

Major security breaches often share common causes despite large budgets and strong teams. These include:

Internet-exposed systems — Sensitive databases or systems should never be directly accessible from the internet or placed in the DMZ.

Ignored security alerts — Warnings about scanning or enumeration activity were often overlooked.

Unpatched vulnerabilities — Systems visible online were left with known, unaddressed weaknesses.

Weak or default credentials — Attackers sometimes gained access simply by logging in with unchanged default passwords or compromised weak accounts.

Key point: Sensitive data should never be stored on systems visible to the public internet

Current Attacks
Current Attacks
WannaCry may be "old" news by now, having had its largest impact between 2017 and 2018; however,
ransomware remains one of the biggest threats to security. WannaCry exploited known vulnerabilities and
crippled many different industries for a time, which taught other hackers that ransomware was not only viable,

but profitable.
Ransomware comes in different flavors and the methods are constantly changing, but the main exploits remain
the same: Gain access, encrypt files and applications, then force the users to pay for the decryption key.
Some ransomware call out to the internet for their keys, some have it pre-installed, and some generate new
keys with every installation. The best way to prevent a large-scale ransomware attack on your network is to
back up everything as often as possible and separate unlike systems to prevent propagation across the entire
network

**Ransomware as a Service (RaaS)**
The idea of Ransomware as a Service (RaaS) is now a possibility, where cyber criminals can create a
customized version of various types of ransomware for profit. On this slide is an example of the Satan
ransomware and the ability to make your own customized version. In this example, 30% of the profits go to
the developer of the service. In other words, you buy a unique instance of an author's ransomware, customized
to your liking, and infect systems with the goal of earning money

**Input Attacks**
Applications often have weak spots because they take input from users — like forms, login boxes, or uploaded files.
Each place where data enters the app is called a trust boundary, meaning the app "trusts" the user's input.
Hackers can send bad or fake data through these entry points to make the app crash, behave strangely, or reveal secrets.
To test this, security testers use a method called fuzzing, where they send lots of incorrect or random data to the app (or a file or network protocol) to see if it breaks or reacts unexpectedly.

**OS Command Injection**

What it is:
OS command injection happens when a web app takes something the user types and directly puts it into an operating-system command. If the app doesn't check or restrict that input, an attacker can add extra commands and make the server run them.

Concrete (simple) example:
A web app runs a shell command to create a folder named after the username. If the app blindly inserts the username into the command, a malicious user could submit:
ksmith; rm -rf /
The shell sees two commands: create-folder ksmith and rm -rf / — the second one would try to delete everything the process can access.

Why it's dangerous:
An attacker can run any command the web app's account is allowed to run — steal data, delete files, install malware, create backdoors, etc.

**OS Command Injection Defenses**

Main idea:
It's dangerous for a web app to run system commands (like with system() in many languages), especially when the command includes user input. That's how attackers can inject extra commands.

Better way:
✅ Don't use system calls if possible.
Most programming languages already have safe functions or libraries to do the same job (like creating folders, copying files, or sending emails).

If you must use system calls:
Be very strict with input:

Only allow safe characters (letters and numbers).

Remove everything else, like symbols ;, &, |, >, etc. — these are meta-characters that let attackers chain commands.

Validate that the input matches exactly what you expect

**Buffer Overflow**

What it is (simple):
A program sets aside a fixed-size chunk of memory called a buffer to hold input (like a password or a message). If the program copies more data into that buffer than it can hold, the extra bytes spill over into nearby memory — that's a buffer overflow.

Tiny analogy:
Think of pouring water into a cup. If you keep pouring past the cup's rim, the water spills onto the table and can knock over other things. The extra input "spills" into other memory and can break the program or change what it does.

Why it's dangerous:

The program can crash (denial of service).

An attacker can craft input so the overflow overwrites control data (like a return address) and make the program run attacker-chosen code. That can give the attacker remote control of the machine.

It often happens in low-level languages (C/C++) that don't automatically check sizes.

Simple example (no code):
Program expects a username of up to 16 characters and allocates space for 16. If someone sends 64 characters and the program blindly copies them, the extra 48 characters overwrite adjacent memory — possibly changing program flow.

How to prevent it (checklist):

Validate input size before copying — always check lengths.

Use safe APIs (functions that take a maximum size) instead of unsafe copy functions.

Prefer memory-safe languages (e.g., Rust, Java, Go) when possible.

Enable compiler/runtime protections: stack canaries, ASLR (address space layout randomization), NX bit (non-executable stack).

Keep libraries up to date and avoid homemade memory handling code.

Fuzz and test inputs to find overflow cases before attackers do.

Fail safely: if input is too large, reject it or truncate in a controlled way.

Normal Stack Memory Allocation (2)
Next, we jump down a line to:
strcpy(buff, input1);
The strcpy() (string copy) function is a deprecated function used to copy text data from one location to

another. The problem with strcpy() is that it does not offer the option to check the length of the user input. In
this case, data being typed in by the user (input1) is being copied into the allocated buffer. At the bottom, you
can see the input. The user is sending in 12 A's using Python. This will not cause any issues as the buffer is
larger than the data being sent into the program. As you can see on the right, the data is written starting from
the top of the buffer downward. You should see something called the Return Pointer a bit further down
below the A's being copied. This is a very important variable as it is used to return control back to the caller of
this function. To use an analogy, when you rent a car, you agree to return it back to the company from which
the car was rented

**Basic Stack Overflow (1)**

In this example we are changing the number of bytes being sent in from 16 A's to 32 A's. You can clearly see
that we have overwritten the return pointer. A capital ASCII A in hexadecimal is 0x41. The 0x meaning the
base16/hexadecimal numbering system and the 41 representing the value in hex. On the bottom-right of the
screen, you can see the words Segmentation Fault. This is the error message reported if an access violation
occurs within the process, such as that of a return pointer pointing to an invalid location in memory.

**Basic Stack Overflow (2)**

Once an attacker discovers that there is a potential buffer overflow within the application, they would likely
run the program under the context of a debugger. This gives them visibility into the internals of the process
and allows for more detail around what is happening. The attacker would determine the number of bytes
necessary to reach the return pointer and overwrite it with an address that points to something more useful.
This could be an address that allows them to bypass authentication, or even the address of intentionally
inserted malicious code that the attacker wants executed

Buffer Overflow Defenses

Run the latest versions of all your installed software
Update and patch your software
Update and patch your languages/runtime environment/server add-ons
Run a vulnerability scanner against your applications
Utilize endpoint protection suites offering exploit mitigations
Validate and sanitize user input

**SQL Injection**

What it is (simple):
SQL injection happens when a web app builds a database query using user input without properly handling it. If the app blindly inserts that input into the SQL string, an attacker can add extra SQL and make the database run commands the app never intended.

Short concrete example (conceptual, no code):
An app asks for a UserID and builds a query like:
SELECT * FROM users WHERE userid = 'USER_INPUT'
If an attacker types:
' OR '1'='1 --
then the query becomes:
SELECT * FROM users WHERE userid = '' OR '1'='1' --'
The OR '1'='1' part is always true, so the database may return all users or skip authentication. The -- turns the rest of the original query into a comment so it's ignored.

Why it's dangerous:

The attacker can read, change, or delete data the web app account is allowed to access.

Some databases let injected SQL call OS-level commands (e.g., via dangerous procedures), so SQL injection can lead to full server compromise.

Even if you only get partial output, techniques like blind SQL injection (timing or boolean tests) let attackers extract data slowly.

How attackers try to hide/extract stuff:

Modify the URL, form fields, or POST body.

Use verbose errors or timing to learn the database structure.

Try many variations until they find one that works (guessing).

Simple prevention checklist:

Use parameterized queries / prepared statements — never build SQL by concatenating raw user input.

Whitelist input where possible (e.g., user IDs: only letters/numbers).

Escape input only as a last resort and using DB-specific escaping functions.

Use least-privileged DB accounts — the web app account should have only the permissions it needs.

Disable risky DB features (e.g., xp_cmdshell in SQL Server) unless absolutely required.

Avoid verbose errors for users — log errors internally but show generic messages externally.

Use ORM layers carefully (they help but aren't a silver bullet).

Web Application Firewall (WAF) can help block common injection patterns as an extra layer.

Monitor and log DB queries and suspicious activity; test your app with SQL injection tests/fuzzing.

Keep DB software patched and follow secure configuration guidance from the vendor.

SQL Injection defenses

Main idea: Stop attackers from sneaking SQL into your app by treating all user input as hostile, limiting what the database account can do, and hiding useful error details.

Simple actions (quick checklist):

Validate input (whitelist). Only allow characters and formats you expect (e.g., user IDs: letters+numbers).

Limit length. Set sensible max lengths — many attacks rely on very long strings.

Escape or remove dangerous characters. Watch for ' " \ ; NULL CR LF and similar — either reject them or escape them correctly for your DB.

Use parameterized queries / prepared statements. Don’t build SQL by concatenating user strings.

Use stored procedures or an application layer. Move SQL generation out of web scripts (3-tier architecture) so the web layer can’t directly craft arbitrary SQL.

Least-privilege DB account. The web app’s DB user should only have the permissions it strictly needs (no CREATE/DROP/ALTER, no change to stored procedures).

Hide SQL errors from users. Show generic error messages to users; log detailed errors internally for admins.

Monitor and log suspicious activity. Watch for repeated SQL errors, strange queries, or patterns that look like injection attempts.

Combine defenses. Use a Web Application Firewall (WAF) and regular security testing (fuzzing, pen tests) to catch what code review misses.

XSS Example

Normal Link Example
https://vulnerable.example.com?id=123
Malicious Link Example

https://vulnerable.example.com?id=%3Cscript%3Ealert%28%27xss%27%29%3C%2Fscript%3E
Decodes to
<script>alert('xss')</script>

Step-by-step: how the payload works

Browser requests the URL containing the encoded value. URL encoding replaces special characters with % codes:

%3C → <

%3E → >

%28 → (

%29 → )

%27 → '

%2F → /

Server receives id=%3Cscript... and decodes the parameter to the string <script>alert('xss')</script> for processing.

Why this is dangerous

Executed JavaScript runs in the victim's origin, so it can read/modify the DOM, steal cookies (unless HttpOnly), make requests with the user's credentials, and perform actions on behalf of the user.

Reflected XSS can be delivered via links (phishing) — a victim clicks the link, the malicious script runs in their logged-in session.

Input Attack Defenses Summary
Remember to:
- Only allow specific valid characters from user input to avoid characters that have special meaning in scripting languages
- Be suspicious of all input, including HTTP headers and cookie data
- Validate on the server, not the client
- Use an up-to-date well-validated third-party library of input checking routines to use throughout the application

Don't forget to check for encoded characters using the same or another up-to-date well-validated third-party library

**Common Types of Malware**

Over the years there have been many different types of malware
- Viruses & Worms
- Trojans
- Rootkits
- Ransomware
- Adware and Spyware

**Virus**

A virus is a type of malware, historically destructive in nature, that typically requires user interaction to infect a system

- They require a host file, such as an executable or Microsoft Office macro
- Once executed, they often copy themselves onto file shares or removable media, email copies of themselves, etc., in an effort to spread
- Their goal is often destructive, such as deleting files; however, they can perform whatever actions they were programmed to do by their author

Worm

A worm is historically a self-replicating piece of code that typically carries one or more exploits and payloads

- They often scan systems for vulnerable services, launch an attack, and execute their payloads
- Their goal is often to infect as many systems as possible and set up C2

A Trojan horse is a program that often performs the desired action of the victim, as well as a malicious action

Let's say we took a copy of the network utility program "netcat" and want to make a trojaned copy that opens up a port on the victim…

- If we have the source code, we could add our malicious code to the beginning of the program, or at any other point within it, compile the code, and publish it
- When the victim executes the program, they will get their desired result of using the tool; however, our code will also execute

**What is a rootkit (plain language)**

A rootkit is malware designed to hide itself and give an attacker persistent, covert control of a system. Its goal is not necessarily flashy damage — it wants stealth and long-term access. Because rootkits alter the system's normal behavior, an infected machine is considered in an unknown, untrusted state.

Primary goals of rootkits

Hide attacker artifacts (processes, files, registry entries, network connections).

Subvert security controls in userland and the kernel so detection tools miss them.

Maintain persistent access so the attacker can return.

Blend in by using legitimate frameworks and services (so activity looks normal).

Why PowerShell / .NET matters (living-off-the-land)

Rootkits (and many modern threats) often reuse trusted, built-in libraries and tools — e.g., Windows' .NET System.Management.Automation (the underlying API used by PowerShell). If malicious code calls this API the same way legitimate scripts do, endpoint monitoring that looks only for the presence of PowerShell may miss it. This technique is called living-off-the-land (LOTL).

Types / layers of rootkits

Userland (user-space) rootkits — modify user-mode binaries or libraries, replace tools to hide activity. Easier to write but easier to detect and remove.

Kernel rootkits — modify kernel code, drivers, or kernel data structures to hide processes/files and intercept system calls. Much harder to detect and remove.

Boot/bootloader rootkits — infect the boot process (e.g., MBR, UEFI) so they start before the OS and can remain stealthy even after OS reinstall.

Firmware rootkits — infect firmware (BIOS/UEFI, NIC, RAID controller). Extremely persistent and difficult to eradicate.

Hypervisor/root-of-trust level — advanced attacks that run below the OS (rare, sophisticated).

Why detection is hard

Rootkits change the platform's behavior so integrity checks may be fooled.

Kernel and firmware-level infections can intercept or modify the very tools you'd use to inspect the system.

They can piggyback on trusted services (PowerShell, system drivers), so signatures and simple heuristics may miss them.

Short version (one-paragraph)

PowerShell is like a powerful built-in toolbox inside Windows. System.Management.Automation is the set of tools in that box (the library that PowerShell uses). Attackers reuse those exact same trusted tools to do bad things so their actions look normal — like a thief wearing a security guard's uniform to walk past cameras. When defenders only look for "is PowerShell running?" they can miss malicious activity that uses PowerShell in normal-looking ways. That trick — using the system's own trusted tools to hide — is called living-off-the-land (LOTL).

**What are Adware and Spyware?**

1. Adware = Advertisement Software

Goal: Show you ads (pop-ups, banners, or redirects) to make money for the creator.

Common "trade": You get something free (like a game, music player, or app), and in return, you agree to see ads.

Example: You download a free game, and every few minutes, it shows an ad before you can continue playing.

Sometimes this "trade" is clearly stated (like YouTube ads for free users), but other times, adware hides itself and floods your system with unwanted ads.

2. Spyware = Spy Software

Goal: Secretly collect information about you — what you do, where you go online, even what you type.

More dangerous than adware because it steals your privacy and possibly sensitive data.

It can:

Track the websites you visit.

Collect personal information without asking.

Change your browser settings (like your homepage or search engine).

In the worst case, install a keylogger — a hidden tool that records everything you type, including usernames and passwords.

Simple example

Let's say you download a free "video converter" from a random website.

You install it.

Suddenly, you see:

New ads popping up everywhere.

Your browser homepage changed.

Your system running slower.

Behind the scenes, the software might also:

Track what videos you convert or which websites you visit.

Send that data back to someone who sells it to advertisers (or worse, hackers).

That's adware + spyware working together

**What is Malware Analysis?**

Malware analysis means studying a suspicious file or program to understand:

What it does

How dangerous it is

How to detect or remove it

It helps security teams know how the malware works and how to defend against it.

Traditional Two Stages

Originally, malware analysis had two main parts:

1. Behavioral Analysis

You run the malware in a safe, isolated environment (called a sandbox) to watch what it does.

You look for:

Files it creates or deletes

Registry changes

Network connections it makes

Processes it starts

Think of it like watching a criminal's actions to see what crimes they commit.

2. Code Analysis

Here, you look inside the program's code itself — not just its behavior.

You study the instructions (machine code) to understand how it does what it does:

What programming language or libraries it uses

How it hides itself or encrypts data

What payload it executes

Think of this as reading the criminal's plan or diary to understand their exact steps.

Fully Automated Malware Analysis

Automation of malware analysis is a good way to perform quick analysis at a relatively low cost when compared to a human analyst

Large amounts of malware can be analyzed quickly, allowing human analysts to focus in on the most serious

However,

- Theyareknowntoproducefalse positives
- Itisnormal for programs to contain strings that also reside in malware
- Critical function calls by legitimate programs will result in alerts
- VirtualProtect, VirtualAlloc, WriteProcessMemory, HeapAlloc, etc.

**What is Static Properties Analysis?**

Static properties analysis means examining a suspicious file without running it — kind of like looking at a bomb without touching the trigger.
It's the safe way to learn about malware, since you don't actually execute it.

Key idea

Behavioral analysis → you run the malware and watch what it does.

Static analysis → you don't run it; you just look at its outside and inner details (structure, metadata, text, etc.).

So static analysis is like doing forensics on a file rather than letting it "come alive."

Interactive Behavior Analysis
In order to further learn about the characteristics and intentions of a malware specimen, it may be required to manually perform behavioral analysis

- The fully automated malware analysis may not have been able to answer all of your questions
- Static properties analysis may not be comprehensive due to packing and other protections

By allowing the malware to run in a controlled environment, we can monitor its behavior as well as manually interact

- Registry and filesystem access
- Process behavior and interactions
- Network activity

**What is "Manual Code Reversing"?**

Manual code reversing means analyzing malware code line by line — like a digital surgeon opening up a program to see exactly what it does inside.
It's the most advanced and most difficult form of malware analysis.

Why It's Important

When malware is analyzed automatically, we only see what it does on the surface (e.g., opens a port, creates a file).

But sometimes, the malware hides its true purpose — encrypting code, pretending to be normal, or running only in special cases.

Manual code reversing lets experts:

Dig into the raw instructions to see exactly what the program does.

Find hidden functions (like stealing passwords, encrypting data, or calling a hacker's server).

Understand how it works, so they can create defenses or antivirus signatures.

Tools Used

Disassembler
Turns machine code (like 01010101) into assembly language — the human-readable form that shows what the program does step-by-step.
Example tools: IDA Pro, Ghidra, Radare2

Decompiler
Converts the assembly language into pseudocode that looks like normal programming languages (like C or C#), making it easier to read.
Example tools: Ghidra, dnSpy, ILSpy

Simple Analogy

Think of malware like a locked black box:

Behavioral analysis = shaking the box to guess what's inside.

Static analysis = looking at the box's label and weight.

Manual code reversing = opening the box and reading every part inside.

It's like a doctor:

Behavioral = giving medicine based on symptoms.

Manual reversing = performing surgery to see the real cause of the problem.

**Web Communication Basics**

What is the web?

From a user’s point of view: The web is all the information, websites, apps, and multimedia that we access on the internet through a browser.

From a technical point of view: The web is mainly a way of sharing information using a system called HTTP (Hypertext Transfer Protocol). It's like a “language” that web browsers (like Chrome or Firefox) and web servers (where websites live) use to communicate.

2. How HTTP Works

HTTP is like a set of rules for how browsers and servers talk to each other.

It’s transaction-oriented: This means that a client (you, using the browser) makes a request, and the server sends back a response.

Stateless: After the server responds, it "forgets" the conversation. So, if you make another request later, the server doesn't automatically remember your previous one.

3. Request and Response

Every communication has two main parts:

Client’s Request: This is when your browser asks for something from a server (like a webpage). It includes:

Method: What type of request (GET, POST, PUT, etc.)

Resource: What specific thing you're asking for (like a webpage or image).

Headers: Extra information, like the website you’re trying to access.

Server’s Response: After the server gets the request, it sends back:

Status Line: Tells you whether the request was successful (e.g., "200 OK").

Headers: Extra info about the response.

Body: The actual content you asked for (like the webpage, image, or file).

4. Types of Requests

GET: Ask for data (like loading a webpage).

POST: Send data to the server (like submitting a form).

PUT: Upload files to the server (like publishing a new webpage).

The method you use depends on what you need to do, but it’s up to the website's designer to decide which method to use.

5. Headers

These are like notes that go along with the request or response. They tell the server extra info about the request (e.g., which website you're trying to reach).

The server might ignore headers it doesn’t understand, but usually, they’re important for things like security or making sure the server knows which website to send back data for.

6. Bodies in Requests and Responses

Sometimes, when you send data (like uploading a file), the body of the request or response will carry that data.

Example: When you fill out a form online and click "submit", the browser sends that form data as the body of a POST request to the server.

**Cookies**

**What are Cookies?**

When you visit a website, your browser and the website talk to each other using HTTP.
But HTTP is stateless — it doesn’t remember you.
That means if you visit one page and then another, the server forgets who you are between requests.

Why is that a problem?

Imagine you’re shopping online:

You add items to your cart on one page.

Then you go to the checkout page.

Without memory (state), the website would forget what you added to your cart!

So we need a way for the website to remember who you are between pages.

The Solution: Cookies

A cookie is like a small note that the website gives to your browser to hold onto.
It says something like:

“This is user #123, and they have 2 items in their cart.”

Your browser saves that note and sends it back to the website every time you visit again.
That way, the website remembers you and your actions.

How It Works

The server sends a cookie to your browser using a special header (Set-Cookie).

Your browser stores that cookie.

Every time you visit that website again, your browser sends the cookie back in a request header.

The website then knows:
“Oh, this is the same user who added things to the cart earlier!”

Privacy & Security

Some people worry cookies can “steal” information from your computer —
but that’s not true.

A website can only create a cookie with the information you give it.

It can’t read your files or grab data from your computer.

Cookies can only be sent back to the same website (or its domain), not everywhere.

To keep things safe:

Websites often encrypt cookie data (so no one can read it).

They can mark cookies as secure, meaning they’re only sent over HTTPS (encrypted connections).

Two Types of Cookies

There are two main types of cookies:
Persistent cookies
Non-persistent (session) cookies

They both store information for websites —
but the difference is how long they last and where they are stored.

1. Persistent Cookies

Think of these as “long-term memory” cookies.

Stored: On your computer’s hard drive
Lasts: Even after you close the browser or restart your computer
Expires: Only when a set time/date is reached (or if you delete it)

✅ Used for:

Remembering your login info (so you don’t have to log in every time)

Remembering site preferences or language settings

Tracking your activity on a website (e.g., ads or analytics)

Risk:
Because they stay for a long time, they can be used to track your online behavior, which can raise privacy concerns.

2. Non-Persistent (Session) Cookies

Think of these as “short-term memory” cookies.

Stored: In the computer’s RAM (memory)
Lasts: Only while your browser is open
Expires: As soon as you close the browser

✅ Used for:

Temporary sessions (like keeping you logged in while browsing)

Secure environments (like banking sites)

Shared computers (libraries, cafés, schools)

Downside:
Once you close your browser, all info is gone —

so you may have to log in again next time.

**SSL/TLS Explained Simply**

What it is

SSL (Secure Sockets Layer) and its newer version TLS (Transport Layer Security) are protocols — special rules — used to encrypt data sent between your browser and a website.

Think of it as creating a private tunnel for your data so that no one can spy on what you're sending or receiving.

When you see https:// (instead of just http://) in the address bar, that means SSL/TLS is being used.

How It Works (Step by Step)

Client connects to the server
You type a website address like https://example.com.
Your browser says: "Hey server, let's talk securely!"

Server sends its SSL certificate
The website sends a digital certificate to prove its identity.
(It's like showing an ID card.)

✅ Client checks the certificate
Your browser checks that the certificate:

Is valid (not expired)

Was issued by a trusted organization

Matches the domain name in the address bar

If all is good, the browser trusts the server.

Key exchange happens
Browser and server create and share secret keys using secure math (like RSA or Diffie-Hellman).
These keys are used to encrypt and decrypt messages.

Secure session begins

Now every message between the browser and server is encrypted.
No one else can read it — even if they intercept the traffic.

What SSL/TLS Does Not Protect You From

❌ Hackers already inside the website (if the site itself is hacked).

❌ Phishing websites (they can also use HTTPS).

❌ Malware or viruses on your computer.

❌ Attacks on the web application itself (e.g., SQL injection).

So, SSL keeps your connection safe, but not everything else.

1. Injection

When a website takes user input and sends it directly into a command or database query without checking it.
Attackers can “inject” malicious commands.

Example:
Typing '; DROP TABLE users; -- in a login form could delete a whole database.

✅ Fix: Always validate and sanitize user input.

2. Broken Authentication

When login or session systems are weak, attackers can steal passwords, session tokens, or even impersonate users.

Example:
A website that doesn’t log users out properly — an attacker could reuse your session.

✅ Fix: Use strong password rules, secure session IDs, and multi-factor authentication.

3. Sensitive Data Exposure

When sensitive info (like passwords, credit cards, or personal data) is not properly protected.

Example:
A website sending your password without encryption over HTTP.

✅ Fix: Use HTTPS (SSL/TLS) and encrypt sensitive data at rest and in transit.

4. XML External Entities (XXE)

Some web apps use XML to handle data. If not configured properly, attackers can make the app read private files or run malicious code.

Example:
An attacker tricks the XML parser into reading /etc/passwd from the server.

✅ Fix: Disable external entity processing in XML parsers.

5. Broken Access Control

When users can access things they shouldn't (like admin pages or other users' data).

Example:
Changing the URL from /user/123 to /user/124 and seeing someone else's account.

✅ Fix: Always check user permissions on every request.

6. Security Misconfiguration

When systems are not set up securely — like leaving default passwords, unnecessary services, or revealing too much info in error messages.

Example:
A site showing detailed error messages with database info.

✅ Fix: Use secure defaults, hide system details, and regularly review settings.

7. Cross-Site Scripting (XSS)

When a site allows users to add text or code that runs in other users' browsers.

Example:
A user posts a comment like <script>stealCookies()</script> that steals others' session info.

✅ Fix: Sanitize user input and escape all HTML/JavaScript output.

8. Insecure Deserialization

When an app loads data (like user profiles) from untrusted sources and trusts it blindly, attackers can send modified data that runs harmful code.

Example:
Sending fake serialized data that gives admin rights.

✅ Fix: Don't trust serialized data, and check data integrity.

9. Using Components with Known Vulnerabilities

Apps often use libraries or frameworks. If they're outdated or have known bugs, hackers can exploit them.

Example:
Using an old version of a framework that has a public security hole.

✅ Fix: Keep all software components updated and patched.

10. Insufficient Logging & Monitoring

When apps don't properly record or monitor what's happening.
This makes it hard to detect or respond to attacks.

Example:
An attacker tries 1000 passwords — and nobody notices because there's no log.

✅ Fix: Log all important actions, monitor for suspicious activity, and alert the team.

| # | Risk | Simple Meaning | Example | Fix |
|---|---|---|---|---|
| A1 | Injection | Bad input leads to commands | SQL injection | Validate input |
| A2 | Broken Authentication | Login/session flaws | Weak password system | Use MFA & secure sessions |
| A3 | Sensitive Data Exposure | Data not encrypted | HTTP passwords | Use HTTPS & encryption |
| A4 | XML External Entities | Bad XML config | File access via XML | Disable external entities |
| A5 | Broken Access Control | Wrong permissions | User accessing admin area | Enforce access checks |
| A6 | Security Misconfiguration | Unsafe setup | Default passwords | Secure configuration |
| A7 | XSS | Script injection in browser | Malicious <script> | Sanitize inputs |
| A8 | Insecure Deserialization | Untrusted data runs code | Tampered data | Validate serialized data |

A9 Vulnerable Components Old software Outdated library Keep software updated
A10 Insufficient Logging No attack detection Missed brute-force attack Log and monitor events

**Developing Secure Web Applications (Made Easy)**

When creating web applications, security should not be an afterthought.
It must be built in from the very beginning — during every stage of software development.

1. Build Security into the Software Development Life Cycle (SDLC)

Security isn't something you "add at the end."
You must think about it from the planning → coding → testing → deploying stages.

2. Train Developers

Developers should understand how hackers think and how attacks happen.
They need training on:

Common web vulnerabilities (like those in the OWASP Top 10)

Secure coding practices (e.g., input validation, session management)

Mistakes specific to their programming language or framework

Example:
If developers know about SQL injection, they'll remember to use parameterized queries instead of unsafe string building.

3. Peer Code Reviews

A peer review means other developers read your code before it goes live.
They can catch:

Bugs

Security issues

Bad coding practices

✅ Benefits:

Problems are found earlier

Mistakes that tests might miss are caught

Code quality improves overall

4. Formal and Thorough Testing

Testing isn't just about making sure the app works —
it's also about making sure it doesn't break under attack.

Test for expected input → What normal users would enter.
Test for unexpected input → What attackers might try.

Example:
Try typing special characters, long text, or script tags to see if the app breaks or exposes data.

Include:

Manual tests (human testers try things out)

Automated tests (tools scan for known issues)

Load tests (simulate many users at once to check performance)

And always record results (what was tested, expected vs actual outcomes).

5. Configuration Management & Version Control

To keep track of who changed what — and to avoid chaos — teams use:

Version control systems (like Git)

Configuration management tools

✅ These ensure:

Everyone works on the correct version of the code

No conflicting or outdated code gets deployed

All changes are documented

6. Separate Environments

Never build or test directly on the production (live) system!
You should have three environments:

| Environment | Purpose |
|---|---|
| Development | Where developers write and test code |
| Testing (Staging) | Where testers check for bugs & security issues |
| Production | The live app users actually see |

Also, developers should not have access to make changes directly to production — that's handled by a separate admin or deployment team to prevent mistakes or hidden code.

7. Why All This Matters

Without secure development practices:

Vulnerabilities slip into the code

Sensitive data can leak

Attackers can exploit weak configurations

Embedding security early:
✅ Prevents vulnerabilities
✅ Saves time and cost later
✅ Builds safer, more reliable applications

**Basics of Secure Coding (Simplified Guide)**

Secure coding means writing software that resists attacks and doesn't expose sensitive data — even when things go wrong.

Here's a breakdown of the core principles and what they really mean

1. Initialize All Variables Before Use

Always set a starting value for every variable.

Don't let users or external input define it automatically.

Example (Bad in PHP):

```
$counter++; // Not initialized
```

A hacker could set ?counter=12 in the URL — and now $counter starts at 12!

✅ Good:

```
$counter = 0;
$counter++;
```

Why: Prevents attackers from injecting data through uninitialized variables.

2. Validate All User Input

Never trust user input — check it, clean it, and limit it before using it.

Use whitelisting (what is allowed) instead of blacklisting (what is not allowed).

Remove or escape special characters (like <, >, ', ", ;).

Example:
Validate form fields (email, phone number, IDs, etc.) before processing.

3. Don't Require Admin Privileges

Your application should not need admin or root access on:

The web server

The database

The operating system

✅ Use least privilege — give the app only the access it really needs.
This limits damage if your app gets compromised.

4. Handle Errors Safely

Always check for errors and handle them gracefully.

Don't show real error messages to users (they may reveal sensitive details).

Example:
❌ Bad: "Database connection failed: user=admin, password=1234"
✅ Good: "An unexpected error occurred. Please try again later."

Log detailed errors to a secure log file for developers only.

Disable debug mode in production.

5. Employ the Principle of Least Privilege

Give each user, process, or component only the minimum access needed.

✅ Examples:

A user shouldn't access admin pages.

A web app shouldn't modify system files.

Database accounts should have only SELECT, not DROP privileges (unless necessary).

6. Don't Store Secrets in Code

Never hard-code things like:

Passwords

API keys

Encryption keys

Backdoor access credentials

Instead:

Store secrets in environment variables or a secure key vault.

Don't rely on "obfuscation" (hiding secrets in complex code). Attackers can find them.

7. Use Trusted Libraries and Frameworks

Don't build your own encryption or authentication systems.
Use tested, well-known libraries instead.

✅ Example:

Use bcrypt or Argon2 for password hashing (not your own hash).

Use JWT or OAuth for session management.

Why?
Because homegrown cryptography or authentication is easy to break.

8. Monitor and Patch Vulnerabilities

If your app uses open-source components, you must:

Stay alert for new security advisories

Apply patches and updates quickly

Remove or replace unmaintained libraries

**Web Application Authentication — Essentials**

Authentication verifies who a user is before giving access to resources.
If it's weak, attackers can easily bypass or guess their way in.

1. HTTP Authentication

Authentication data is sent in the HTTP header.

Mode Description Security Risk
Basic Sends username and password encoded with Base64 Easily decoded — not secure without HTTPS
Digest Sends MD5 hash of password Better than Basic, but MD5 is outdated and weak

Always combine with HTTPS (SSL/TLS) to protect credentials in transit.

2. Form-Based Authentication

The most common type of login on websites.

User enters credentials via an HTML form → data sent in cleartext unless HTTPS is used.

Must rely on secure channels (SSL/TLS) to protect credentials.

4. Multifactor Authentication (MFA)

MFA = Using 2 or more factors to prove identity.

Factor Type Description Example

| | | |
|---|---|---|
| Something you know | Knowledge-based | Password, PIN |
| Something you have | Physical/virtual device | Token, phone, smart card |
| Something you are | Biometric | Fingerprint, face scan |

5. Common MFA Implementations

Certificate-Based Authentication

Uses a digital certificate stored on the user's device.

Strong but hard to manage for large systems.

Token-Based Authentication

Uses a hardware or software token to generate a one-time password (OTP).

Examples: Google Authenticator, Authy, Duo, Microsoft Authenticator.

✅ Safer and easier than password-only systems.

6. Out-of-Band Authentication

Sends a one-time code via another channel, like SMS or email.

Used as a second step after password entry.

Adds security — but SMS can be intercepted, so use with caution.

7. Device "Footprint" Authentication

Uses device attributes like:

Cookies from past logins

Browser version / IP address / OS info

Installed local signatures or tokens

Easy to spoof, but useful for lightweight second-factor checks.

8. ❓ Challenge Questions

Example: "What is your first pet's name?"

Weak because answers can often be guessed or found online.

Should not be the only security factor.

Best Practices for Secure Authentication

✅ Use HTTPS for all authentication traffic
✅ Remove or change default admin/test accounts
✅ Implement account lockout after failed logins
✅ Always validate session tokens before serving pages
✅ Require MFA for privileged or admin access
✅ Return generic error messages (“Invalid credentials”)
✅ Log authentication attempts for auditing and monitoring

**Access Control — Keeping Attackers Out**

Even if your authentication is strong, attackers may still explore your website in ways normal users never do.
Access control ensures users — authenticated or not — can’t reach areas they shouldn’t.

1. The Difference Between Users and Attackers

| Normal Users | Attackers |
| --- | --- |
| Follow links and buttons as intended | Manually test hidden or unlinked URLs |
| Stay within the app’s workflow | Try to access backend folders, admin pages, configs |
| Use browser normally | Use tools (Burp Suite, curl, scripts) to find vulnerabilities |

Goal: Don’t assume users will only take the "intended" path through your site.

2. Protect Hidden or Sensitive Areas

Attackers love to explore:

Default pages and sample sites (e.g., /index.html, /test.php, /examples/)

Code library files (e.g., /includes/functions.php, /config.php)

Configuration files (.env, web.config, .htaccess)

Old backups or test folders (/backup/, /old/, /temp/)

✅ Prevention Tips

Delete or restrict default / sample content.

Move code libraries and configs outside the web root.

Use file permissions and deny public access to sensitive files.

**Prevention:**

Validate and sanitize all URL input.

Never use user input to build file paths directly.

Configure the web server to restrict path access (chroot, sandbox, etc.).

Use least privilege for the web server account.

5. Defense-in-Depth Checklist

| Risk | Defense |
|---|---|
| Default/sample pages | Remove or restrict them |
| Code library exposure | Move outside public directory |
| Directory browsing | Disable in web server settings |
| Config files exposure | Restrict via .htaccess / permissions |
| URL traversal | Validate and sanitize inputs |
| Unintended file access | Enforce least privilege and proper ACLs |

**HTTP is stateless (the main problem)**

HTTP does not remember anything.

Every time your browser sends a request to a website, the server treats it as a new request.

The server doesn't know if two requests came from the same user.

Example:
If you log in and then open another page, the server would forget you logged in unless something is used to remember you.

2 Why do applications need sessions?

Web applications need to:

Know who the user is

Remember things like:

Login status

Shopping cart items

User preferences

This is called a session.

3 How do applications remember users?

They use Session IDs (the most popular method).

A Session ID is:

A unique random value (like: A9X3K2Q7)

It represents your session, not your password

4 How Session IDs work (step by step)
Step 1: Session starts

You open a website (or log in).

The server creates a unique Session ID for you.

The server sends this Session ID to your browser.

Step 2: Where is the Session ID stored?

The browser stores the Session ID in one of these places:

Cookie (most common)

Hidden form field

URL query string
Example:

https://example.com/page?sessionid=12345

Step 3: Session continues

Every time your browser sends a new request:

It automatically sends the Session ID back to the server.

The server uses the Session ID to:

Identify you

Load your session data (login, cart, settings)

Why this is dangerous ✕

Anyone can see the Session ID in the URL

The user can:

Edit the URL

Replace the sid with another value

If the server doesn’t check properly, the attacker may access another user’s session

Example attack

Change:

sid=34112323

To:

sid=99988877

➡ If valid → attacker sees another user’s account

2 Hidden Form Elements
What it is

The Session ID is stored inside the HTML code

It is not visible on the web page, but exists in the source code

Example
<INPUT TYPE="HIDDEN" NAME="Session" VALUE="22343">

Why this is risky ✕

The user can:

View page source

Save the page to their computer

Change the Session ID value

Using a proxy tool (like Burp Suite), the Session ID can be:

Modified while the request is being sent

3 Cookies (Most Common Method ✅)
What it is

The Session ID is stored as a cookie in the browser

The browser automatically sends it with every request

Example (conceptual)
SessionID=ABCD1234

Security risks ❌

The cookie can be:

Edited on the hard drive

Modified on the fly using a proxy

If stolen → attacker can hijack the session

4 Using a Proxy (On-the-fly modification)

A proxy (like Burp, OWASP ZAP) allows an attacker to:

Intercept requests

Modify:

Session IDs

Cookies

Hidden fields

Forward the modified request to the server

Protection from Session Attacks (Easy Explanation)

Most session attacks happen because an attacker:

Steals a Session ID

Or guesses a valid Session ID

So the main goal is:
Make session IDs hard to guess, hard to reuse, and short-lived

1 Make Session IDs random and long
Why?

If Session IDs follow a pattern, attackers can guess them

Short Session IDs can be brute-forced

Best practice

Use:

Long Session IDs

Truly random values (not sequential numbers)

✅ Example (good):

aF9xQ2Lk8MZ7P1Y4R

❌ Example (bad):

1001, 1002, 1003

2 Don't build your own session system ❌
Use a trusted framework instead ✅
Why?

Session handling is hard to secure

Established frameworks:

Have been tested for years

Fix known vulnerabilities

Examples

Django sessions

Spring Security

ASP.NET session management

PHP built-in session handling

Never “home-grow” your own session logic

3 Test Session ID predictability
Why?

Developers may think IDs are random, but they aren’t

How?

Use security tools like:

Burp Suite

OWASP ZAP

These tools:

Collect many Session IDs

Analyze patterns

Show if IDs are predictable

4 Digitally sign or hash Session IDs
What does this do?

Ensures the Session ID:

Was created by the server

Was not modified by the attacker

Result

If an attacker changes the Session ID:

The server detects it

The request is rejected

5 Only send the Session ID — nothing else
Bad practice ❌

Sending sensitive data to the browser:

Account number

Shopping cart

User role

Because:

Anything on the client can be:

Read

Modified

Stolen

Best practice ✅

Send only the Session ID

Store all other session data:

On the server

In a database

Indexed by the Session ID

6 Encrypt session data if it must be stored on the client
Sometimes needed

User preferences

Language settings

Rule

If session data is stored in:

Cookies

Hidden fields
➡ Encrypt it

This prevents:

Reading the data

Modifying the data

7 Regenerate Session ID after login
Why?

Prevents Session Fixation attacks

How it works

User visits site → gets anonymous Session ID

User logs in

Server generates a new Session ID

Old Session ID is destroyed

8 Expire Session IDs
Two important rules

On logout:

Session ID becomes invalid immediately

On inactivity:

Session ID expires after a short timeout

Why?

Prevents attackers from reusing old or stolen sessions

9 One-page SOC summary

Session IDs must be:

Random

Long

Unpredictable

Use trusted session frameworks

Pass only Session ID to the client

Encrypt any client-stored session data

Regenerate Session ID after authentication

Expire sessions on logout or timeout

**What is Web Application Monitoring?**

Web Application Monitoring is the continuous process of watching a web application to detect attacks, failures, misuse, or abnormal behavior.
Because internet-facing web applications are constantly targeted, administrators cannot rely on assumptions—they must actively monitor to know if attacks are happening and whether they succeed.

Key Goals of Web Application Monitoring
1. Monitor Web Content and File Integrity

This ensures that no unauthorized changes occur to:

Web pages

Configuration files

Application code

Why this matters:

Attackers often deface websites to spread messages or gain reputation.

More dangerous attacks make subtle changes, such as:

Modifying login forms to steal credentials

Redirecting users to malicious sites

Adding hidden phishing pages on your server

How it's done:

Content monitoring: Periodically load web pages and compare them to a known-good version.

File Integrity Monitoring (FIM): Alerts when files are added, modified, or deleted outside approved rules.

2. SIEM Correlation Is Critical for Timely Detection

A SIEM (Security Information and Event Management) system collects and correlates logs from:

Web servers

Firewalls

Databases

IDS/IPS

Why correlation matters:

A single log entry may look harmless.

Correlated events can reveal:

Web attack attempts

Successful exploitation

Lateral movement or data exfiltration

Example:

Multiple failed logins → followed by a successful login → followed by file changes

This pattern strongly indicates compromise.

3. Check Availability of Web Application Components

Monitoring must confirm that the application is up and usable.

What can cause downtime:

Hardware failure

Software bugs

Network issues

Denial-of-Service (DoS) attacks

Basic monitoring:

Load a webpage and alert if it fails

Advanced monitoring:

Log in as a real user

Perform a transaction or database query

Verify the expected result

This tests the full application stack, not just the web server.

4. Track Performance, Trends, and Anomalies

Monitoring performance helps detect:

Slow response times

Resource exhaustion

Abnormal traffic spikes

Why this is important:

Performance degradation can signal:

Application bugs

Database issues

Ongoing attacks (e.g., DoS, brute force)

Tracking trends helps you identify abnormal behavior early.

5. Examine Web Server Log Files Regularly

Web server logs contain critical evidence, such as:

Suspicious URLs

SQL injection attempts

Directory traversal attacks

Unusual HTTP status codes

Regular log review helps:

Detect attacks in progress

Investigate incidents

Feed data into SIEM for correlation

6. Scrutinize User-Provided Content Areas

Areas that accept user input are high-risk, such as:

Comment sections

File uploads

Forms

Forums

Why extra scrutiny is needed:

These areas are common targets for:

Cross-Site Scripting (XSS)

File upload attacks

Web shells

Phishing page hosting

7. Verify Backend Database Security

The web application is only as secure as its backend database.

Monitoring ensures that:

Databases are not directly exposed to the internet

Access controls are enforced

Abnormal queries or access attempts are detected

Compromised databases often mean data breaches.

Monitoring Implementation Approaches
1. Commercial Monitoring Services

Hosted by third-party providers

You configure checks via dashboards

Often include:

Defacement detection

Availability monitoring

Multi-step login tests

Pros: Easy to deploy
Cons: Ongoing cost, less customization

2. On-Premise Commercial Software

Installed on your own servers

Greater control over data

3. Custom (Home-Grown) Monitoring Solutions

Built using scripting languages like Python or Perl

Highly flexible

Can perform custom logic and alerts

Pros: Maximum flexibility
Cons: Requires skill, time, and maintenance

Monolithic Architecture and Security Controls
What Is Monolithic Architecture?

A monolithic application architecture is a design where all application functionality exists in a single tier or single codebase.
This includes:

User interface logic

Business logic

Authentication and authorization

Data access logic

Everything runs together as one application rather than being split into microservices.

Trust Boundaries in a Monolithic Architecture

A trust boundary is any point where data or traffic moves between components with different trust levels.
In a typical internet-facing monolithic web application, the main trust boundaries are:

Client browser → Load balancer / web tier

Web application → Database

Security controls are applied at each of these boundaries.

Security Controls by Entry Point / Trust Boundary
1. Client Browser to Load Balancer (Edge Security)

Traffic flow:

Clients connect over HTTPS (TCP port 443)

All other ports are blocked

Security controls at this layer:

TLS/HTTPS encryption to protect data in transit

Firewall rules to restrict allowed ports

Web Application Firewall (WAF)

WAF capabilities include:

Rate limiting (mitigates brute force and DoS attacks)

Blocking automated scanners and bots

Denying known malicious IP addresses

Detecting and blocking common web attacks (e.g., SQL injection, XSS)

This layer focuses on filtering and controlling traffic before it reaches the application.

2. Web Application Tier (Application-Level Security)

In a monolithic architecture, the web application tier handles all security logic, including:

Authentication (verifying user identity)

Authorization (what the user is allowed to do)

Access control and entitlements

Input validation

Output encoding

This is where the OWASP Top Ten risks primarily occur, such as:

Injection attacks

Broken authentication

Broken access control

Security misconfigurations

Because everything is centralized, any security weakness here affects the entire application.

3. Database Tier (Data Security)

The database tier is responsible for protecting sensitive data, including:

Encrypting data at rest

Enforcing strong authentication for client connections

Ensuring data in transit is encrypted (TLS)

Security controls here prevent:

Unauthorized database access

Data leakage

Credential abuse from compromised application components

Security Advantages of Monolithic Architecture

Even though security controls are often missing or poorly implemented, monolithic architectures have one key advantage:

Centralized Security

Security logic exists in very few locations

Easier to:

Audit security controls

Monitor behavior

Apply patches and fixes

Fewer integration points compared to distributed architectures

This centralization is important because security failures are easier to identify and correct than in highly distributed systems.

Microservice Architecture (Security Perspective)
What Is Microservice Architecture?

A microservice architecture breaks a large, monolithic application into many small, independent services, each responsible for a single business function.

In this model:

A modern HTML5 single-page web application (SPA) calls REST APIs

Mobile applications consume the same APIs

Even IoT devices (like the fictional Dunder Mifflin paper-ordering device) interact with microservices

All of these clients communicate over the network, usually using HTTP/HTTPS and JSON APIs.

Functional Decomposition

Each microservice represents a single business capability, such as:

Human Resources

Customer Service

Employee Management

Account Management

Discount / Coupon Services

Unlike monolithic apps:

Each microservice is deployed independently

Each may use different backend data sources, such as:

Databases

Object storage (S3)

Block storage (EBS)

This dramatically increases the number of trust boundaries.

CQRS in Microservices

Some architectures further divide services using Command Query Responsibility Segregation (CQRS):

Read APIs → Retrieve data

Write APIs → Create or update data

Delete APIs → Remove data

Security Goal of CQRS

Isolate high-risk operations (e.g., delete)

Reduce the blast radius of a compromised API

Security Risk of CQRS

Adds operational and architectural complexity

More services = more:

APIs

Authentication paths

Configuration errors

As Martin Fowler notes, complexity itself can weaken security if not carefully managed.

Security Challenges in Microservice Architectures

Compared to monolithic systems, microservices introduce:

Many more entry points

Multiple client types (browser, mobile, IoT)

Extensive east-west traffic (service-to-service)

Decentralized authentication and authorization

Security is no longer centralized—every service becomes a potential target.

As a Security Professional: Where Do You Start?
1. Secure the Edge (Client → API)

Start with all external entry points:

API gateways

Load balancers

Public endpoints

Apply:

TLS everywhere (HTTPS only)

API authentication (OAuth2, tokens, mTLS)

Rate limiting

WAF rules for API attacks

Bot and abuse protection

This is the first and most exposed trust boundary.

2. Enforce Identity and Access Between Services

Each microservice must:

Authenticate other services

Authorize requests explicitly

Use:

Service identities (not shared credentials)

mTLS or short-lived tokens

Least-privilege access policies

Never assume internal traffic is trusted.

3. Secure Each Microservice Individually

Every service must implement:

Input validation

Output encoding

Authorization checks

OWASP Top Ten protections

There is no “one security layer” anymore.

4. Protect Backend Data Sources

For each database or storage system:

Enforce authentication

Encrypt data at rest

Encrypt data in transit

Restrict access per microservice

A compromised service should not expose all data.

5. Centralized Logging, Monitoring, and SIEM

Because security is distributed, visibility must be centralized:

Aggregate logs from all microservices

Correlate API calls, failures, and anomalies

Detect lateral movement and abuse patterns7

Security Operations and
Log Management

One of the challenges in cybersecurity is timely detection of attacks. Adversaries compromise systems for a
long time before an organization detects it. While there are many reasons for this, one of the primary reasons
is not having proper visibility to see what is happening. Logs contain events that ultimately are the evidence so
you can track, find, and monitor the adversary. Logs provide the optics that are required for timely detection.
While turning on logging is important, it is just as important to make sure that you have the right logs

Core Logging Terms

1. Message

A message is any system-generated indication that an event has occurred.

Examples:

User logged in

File accessed

Service started or stopped

A message is the raw event information before it is stored or acted upon.

2. Log File

A log file is a collection of messages stored for later review.

Examples:

Web server access logs

Firewall logs

Authentication logs

Log files provide historical evidence of system activity.

3. Alert

An alert is a special type of message designed to notify an operator of an important or suspicious event.

Examples:

Multiple failed login attempts

Malware detected

Disk space critically low

Alerts demand attention or action, unlike normal logs.

4. Device

A device is any system that generates security-relevant logs.

Examples:

Firewall

Server

Router

Endpoint

Application

In SIEM terms, the device is the log source.

Additional Important Logging Terms
5. Logging

Logging is the act of creating log records.

This includes:

Writing messages to files

Sending logs to a SIEM

Recording system activity

6. Auditing

Auditing is the process of reviewing log records to verify:

Policy compliance

Proper usage

Security controls effectiveness

Auditing is usually periodic, not real-time.

7. Monitoring

Monitoring means real-time or near real-time observation of logs and events.

Purpose:

Detect attacks quickly

Identify failures as they happen

Monitoring is proactive, unlike auditing.

8. Alerting

Alerting is the act of sending notifications to operators when certain conditions are met.

Examples:

Email alerts

SIEM notifications

Pager or SMS alerts

Important note:

An alert itself is also treated as a log entry.

How to Start Managing Logs (Easy Explanation)

Most companies don’t manage all logs at once.
They usually start simple and improve over time.
Understanding this order helps you know where to begin and why.

1 Start with Firewall Logs (Easiest)

Firewalls are usually the first thing companies monitor.

Why?

Firewall logs are:

Easy to collect

Easy to understand

They usually use syslog or key-value format

They show:

Allowed traffic

Blocked traffic

Source and destination IPs

Firewall logs = best starting point

2 Then Look at NIDS Logs (Second Priority)

NIDS (Network Intrusion Detection System) logs show attack attempts.

Why they are useful:

Detect scans, exploits, and suspicious traffic

Why they are harder:

Many false positives

Not every alert is a real attack

Even with noise, NIDS logs are very valuable and should be:

Centralized

Reviewed with firewall logs for context

3 Operating System (OS) Logs Came Later

Originally, OS logs were reviewed only when something broke.

Now:

We collect OS logs continuously

Tools make it easier using:

Agentless collectors

Unified log formats

OS logs help detect:

Unauthorized logins

Privilege abuse

System changes

4 Web Server Logs (Often Ignored at First)

Web server logs were not originally used for security.

They were mainly used to:

Count visitors

Measure website success

Problems in the past:

Logs stored only locally

No easy forwarding

Hard to review or back up

Because of this, security teams often ignored them early on.

Log Filterin
Not all logs are useful.
Some help detect attacks, some are for compliance, and some are just noise.
So organizations must decide what logs to keep and what to ignore.

How to Start Using the Tools
Here we learn how to start using the tools for taking control of your logs.
Start by collecting logs; use syslog-ng or whatever syslog variant is available on your systems. To combine
these with Windows logs, use NXLog or Winlogbeat.
Store logs in files (compressed or not) or a log storage system such as Elasticsearch.
To start peeking at logs, use search logs like free "grep", "Kibana", or "Splunk" (previously mentioned).
When ready to move to correlation and alerting, get OSSEC or other tools. At this point, you gain a degree of
awareness of what is going on in your environment

**Log Monitoring Strategy — Simple Summary**

Log monitoring is about watching logs regularly (not always daily) to spot suspicious activity.
At first, we usually don't know if something is bad — we just know it looks interesting or unusual.

Simple Monitoring Flow

1 Notice something suspicious

An unusual log entry

Something that caused problems before

2 Ask: Is this a known bad event?

Yes → Start incident response

Example: confirmed attack, compromised system, traffic to a known malicious site

No → Investigate more

3 Investigate unknown events

Decide if it is:

Normal behavior (OK)

A real security problem (incident)

4 Handle false alarms

If it turns out to be harmless:

Tune rules

Adjust alerts

Reduce false positives

Key Idea (Very Important)

Logs are not "good" or "bad" at first — they are interesting, suspicious, or normal.

**Setting Up a Log Monitoring Program — Simple Phased Summary**

You cannot monitor all logs at once.
Log monitoring must be built step by step (phased approach) because collecting logs from many systems takes time and integration effort.

Phase 1: Choose Which Logs to Collect First

Most organizations follow a natural order (because it's easier):

1Firewalls & network devices
2 Network security tools (IDS/IPS)
3 Servers (Linux/UNIX → Windows)
4 Server applications (web, mail)
5Databases
6 Applications
7Desktops/endpoints

This order can change depending on goals

Internal threat focus → start with servers

External threat focus → start with firewalls

Phase 2: Decide Where to Start in the Network

Choose which network areas to connect first:

Common approach: DMZ → core → internal

Another approach: internal systems first (when application auditing is the priority)

Phase 3: Grow the Program Gradually

Over time, the program expands in three ways:

More log sources

More network segments

More users

Example user growth:

Security team → IT teams → auditors

❌ Avoid each team running its own log system
✔ One centralized log platform is more efficient and cheaper

Phase 4: Add More Use Cases

Once logs are collected, they are used for:

Availability monitoring

Incident response

Compliance

Other business needs

There is no fixed order here — it depends on the organization.

**Challenges to Organization-Wide Log Management — Simple Summary**

When deploying log management across an entire organization, the problems are not only technical.
Most challenges come from people, rules, and logistics.

Main Challenges (Explained Simply)
1 Organizational Politics (Biggest Problem)

Different teams own different systems

Not everyone wants to share logs

Many groups must send data to one central system

This makes log management an integration and coordination challenge, not just a technical one.

2 Legal and Data Privacy Issues

Logs contain sensitive data

Moving logs across countries may break laws

Example:

Can a U.S. analyst monitor logs from France?

Because of privacy laws, legal teams must be involved, especially for global deployments.

3 Remote Location Access

Log sources may be in remote offices or countries

Central management is possible, but:

Installation often needs local access

Requires:

Travel

Or cooperation from local system administrators

4 Custom and Unsupported Log Formats

Some applications generate:

Custom

Poorly documented

Unsupported logs

Good news ✅:

Modern log management tools can still collect and process all logs, even custom ones

**Log Aggregation and SIEM — Simple Summary**

Log aggregation means collecting all logs in one place.
SIEM is the tool that analyzes and connects those logs to detect problems or attacks.

Simple Steps Explained
1 Centralize Your Logs

Collect logs from all systems into one location

This makes monitoring and investigation easier

2 Keep Original Logs Safe

Always keep unchanged (original) logs

Some logs are needed for legal or compliance reasons

Normalized logs may not be valid in court

Best practice:

One system to store original logs

Another system to analyze and correlate logs

3 Normalize Logs

Logs come in different formats

Normalization makes them look the same

This allows easy searching and comparison

4 Correlate Logs

Compare logs from different systems

Look at events that happened around the same time

Example:

Firewall allows traffic

Web server receives request

Database shows access
→ Together, this tells a full story

5 Use the Same Time Everywhere

All systems must use the same time source

This avoids confusion when matching events

✔ Best practice:

Use a central time server (NTP)

Use UTC if systems are in different countries

6 Detect Log Deletion or Tampering

SIEM should detect:

Deleted logs

Modified or corrupted logs

Missing logs can mean an attacker is hiding activity

What Problem Does Sigma Solve?

Every SIEM (Splunk, QRadar, Sentinel, etc.) uses different rule formats

This makes rules hard to share or reuse

Sigma fixes this by acting as a common language.

How Sigma Works (Very Simple)

1 Write a detection rule in Sigma format
2 Use a tool called sigmac
3 sigmac converts the rule into:

Splunk rule

QRadar rule

Elastic rule
(depending on the SIEM you use)

onversion of Signatures to Alert Queries
To make Sigma rules work with a SIEM, they must be converted using sigmac. The process looks as follows:
1. Write rule in Sigma Format
2. Create a file mapping generic field names to field names within a given SIEM platform
3. Use sigmac to convert from Sigma format to a specific SIEM product rule format

Because Sigma rules are a generic format, they are perfect for sharing. Projects such as MISP make this a

simple task. Now, organizations can create custom rules and share them openly with anyone, or strategically
with specific partners or industries.
However, sharing Sigma rules introduces new problems: A rule that works in one environment may not work
well in another. Some rules are intended to be applied based on knowing thyself while others map to attacker
methodologies. Therefore, rules that are shared should go through a quality review

Orchestration
This slide represents a possible automated quality review and orchestration for implementing shared rules. The
steps would be automated with a script and perform as follows:
1. Pull shared rule from MISP.
2. Convert Sigma rule with sigmac to a SIEM-specific format.
3. Test rule outside alert engine by seeing if it would match on existing data.
4. If matches are found, move to a location for manual assessment and open a ticket. Otherwise, validate
performance and, if good, move alert into production.
The process above is an example of how rule validation and sharing can be validated and could have any
number of logical evaluations.

Real-time log monitoring focuses on detecting critical security and operational events that require immediate response. Key situations include:

Malware outbreaks: Detected via antivirus, firewall, or IDS logs; require quick quarantine and cleanup, especially since AV tools can miss threats.

Confirmed intrusions: Strong evidence usually comes from correlating multiple logs (IDS/IPS + firewall) and often demands rapid system shutdown or isolation.

Serious internal abuse or policy violations: Such as access to illegal content, typically identified through web proxy logs, with high legal risk.

Regulated data theft: Triggers urgent action due to direct damage and regulatory penalties.

Loss of critical services: Caused by attacks, human error, or bugs; server logs help detect and respond quickly.

Log deletion or corruption: Indicates possible intrusion or insider abuse and must be investigated immediately.

Overall, these events represent high-risk scenarios where fast detection and response are essential to limit damage

Summary – Daily Tasks in Log Monitoring

Daily log monitoring focuses on identifying issues that require a response within the same day, even if they are not immediately critical. Key tasks include:

Unauthorized configuration changes: Detecting unapproved changes to system or network settings is essential for maintaining control and security.

Disruptions in non-critical services: Errors in secondary or less important systems (e.g., backup DNS servers) should be addressed promptly to prevent escalation.

Weaker intrusion indicators: Patterns like many attacks from a single source, even without confirmed success, require investigation.

Suspicious login failures: Multiple failed logins across systems or followed by a successful login may indicate external probing or insider threats and need quick response.

Minor malware activity: Spyware infections or antivirus failures should be handled within a day.

Daily activity summary reports: Reviewing consolidated log reports improves situational awareness and helps spot abnormal behavior.

Summary – Weekly Tasks in Log Monitoring

Weekly log monitoring focuses on trend analysis and issues that typically require a response within a week. This timeframe helps identify patterns that are not visible daily. Key tasks include:

Trend analysis of logs: Review summaries of events over time to spot abnormal patterns, such as:

System failure trends

Attack trends

Configuration change trends

User access trends

Use of reporting tools: Commercial log management systems often provide these reports; custom systems should support similar summaries.

Account management review: Check for unauthorized account creation and ensure accounts of terminated employees are removed, using HR data when possible. While reviewed weekly, real-time or daily monitoring is ideal for faster detection.

Host and network device changes: Review system and network configuration changes to distinguish normal authorized activity from unauthorized modifications.

Summary – Monthly Tasks in Log Monitoring

Monthly log monitoring focuses on long-term patterns, policy compliance, and performance evaluation. Key tasks include:

Long-term trend review: Analyze monthly system and network log trends to understand normal behavior patterns (e.g., weekend slowdowns, end-of-month activity) and identify anomalies. Monthly maintenance activities are often visible here.

Minor policy violations: Review logs (firewall, web proxy, IDS/IPS) for acceptable use policy (AUP) violations or inappropriate web browsing.

Resource usage analysis: Examine reports on bandwidth usage, per-user or per-application consumption, and resource exhaustion issues (e.g., high memory or CPU load).

Security control effectiveness: Use logs to measure security performance, such as successfully blocked attacks, denied firewall connections, and other indicators that security tools are working as intended.

Simple Summary – Quarterly Tasks in Log Monitoring

Quarterly log monitoring focuses on proof, long-term patterns, and keeping the logging program up to date. In simple terms:

Create audit reports: Prepare reports that prove daily and weekly log reviews were actually done. This is required for compliance (for example, PCI DSS).

Review long-term behavior: Look at log trends over several months to understand normal quarterly activities (like increased system usage during quarterly business reporting).

Save evidence: Keep quarterly reports as proof in case auditors ask for them later.

Update log coverage: Review infrastructure changes to ensure new systems, devices, or applications are included in log collection.

Adjust for new rules: Update log monitoring if new regulations or compliance requirements appear.

In short:
Quarterly tasks are about showing compliance, understanding long-term normal activity, and making sure logging still covers everything it should.

Simple Summary – Annual Tasks in Log Monitoring

Annual log monitoring is about reviewing the big picture and planning for the future. In simple terms:

Review logging policy: Check if you are collecting too many logs (noise) or too few logs (missing important data).

Verify log retention and backups: Make sure archived logs can actually be restored and read, even after system or vendor updates.

Review long-term trends: Analyze logs over the entire year to spot major patterns, growth in log volume, and recurring issues. Save these reports for future reference.

Support budget planning: Use logs to understand system capacity, resource exhaustion, and bandwidth usage to plan next year's IT and security budget.

Check new regulations: Review any new or updated compliance requirements that may affect what logs must be collected or how long they must be kept.

In short:
Annual tasks help ensure logging is effective, compliant, reliable, and useful for long-term planning.

How Logs Help (and Their Limits)

Logs are useful because they help answer key questions about an event:

Who did it

Where it came from

When it happened

How it happened

What happened

However, logs are not always perfect or complete, and each question has limitations:

Who: Logs may show a username or system account, not necessarily the real person.

Where: IP addresses or locations can be spoofed, so location data may be misleading.

When: Time issues matter—wrong time zones or incorrect system clocks can confuse timelines.

How: Logs often suggest how something happened, but they may reflect assumptions rather than proven facts.

What: Logs only show what was recorded, not everything that actually happened.

In short:
Logs are essential for investigations, but they must be analyzed carefully, correlated with other logs, and not trusted blindly as absolute truth.

# Digital Forensics

## What Is Digital Forensics?

Digital forensics is the science of finding, collecting, and analyzing digital data in a careful and legal way.

According to NIST, it means:

Identifying digital evidence

Collecting it properly

Examining and analyzing it

Preserving data integrity

Keeping a strict chain of custody so evidence can be trusted in court

In simple terms:
Digital forensics is used to investigate data from computers, phones, storage devices, cloud systems, and IoT devices.

Where it started:

Military and law enforcement investigations

Devices seized during arrests contained valuable evidence

How it evolved:

Started with PCs and floppy disks

Expanded to smartphones, cloud environments, and IoT systems

In short:
Digital forensics helps investigators find the truth in digital data while ensuring the evidence remains valid and trustworthy.

Who Uses Digital Forensics & Incident Response (DFIR)?

Digital forensics and incident response are used by different groups, each with a specific goal:

InfoSec (Security) Teams

Detect and respond to security breaches

Work inside organizations or as external consultants

Focus on protecting systems and reducing damage

DoD / Intelligence Community

Identify and eliminate threats

Analyze attacker tools, methods, and behavior

Support national defense and intelligence operations

Law Enforcement

Investigate crimes and attackers

Support national security efforts

Work with prosecutors when cases go to court

Legal Support

Use digital evidence to prosecute or defend cases

Handle criminal and civil investigations

Often focus on eDiscovery (finding and managing digital evidence)

In short:

Digital forensics is used by security teams, military and intelligence agencies, police, and legal professionals to investigate incidents, understand attackers, and support legal or security actions

Investigative Process in Digital Forensics

Digital forensics is like being a detective for computers and devices:

Follow the evidence: Let the logs and data show what happened.

Be objective: Don’t jump to conclusions. Stay neutral.

No assumptions: Only trust what the evidence actually proves.

Protect evidence: Handle data carefully so it stays trustworthy.

Think scientifically: Test ideas against the evidence, like the scientific method.

In short:
Digital forensics is about finding the truth from evidence, carefully and objectively.

“Follow the Evidence” in Digital Forensics

Digital forensics follows a step-by-step approach to trace what a user did on a system:

User Activity – What the person did:

Web browsing

Downloading files

Connecting USB drives

Visiting malicious websites

Creates Evidence – What is left behind:

Browsing history

Disk artifacts (files, shortcuts)

Event logs

Network artifacts

Recovered by Digital Forensics – How investigators analyze it:

Data carving and recovery

Packet/network traffic analysis

Artifact parsing

Creating timelines of activity

Malware analysis

How to Keep Digital Evidence Safe

Don’t Change Anything:

Every time you touch a device, you might leave traces. Try to avoid altering it.

Write Everything Down:

Chain of Custody: Track who has the evidence and when.

Record device details (like model and serial number).

Note the time you start and finish, and any commands you type.

Use Safe Tools:

Write-blockers: Let you read the data without changing it.

Read-only access: Only view the files.

Keep evidence in safe storage.

Main Idea: Be careful, document everything, and don't change the evidence.

Examples of Digital Forensic Artifacts

What they are:
Artifacts are digital traces left behind on a computer that show what happened. They help investigators understand system activity. New artifacts are discovered all the time, so this is always evolving.

Common types of artifacts:

File Download (Purple): Shows what files were added to the system.

Program Execution (Red): Shows which programs were run.

File/Folder Opening (Green): Shows which files or folders were opened.

Deleted Files or Metadata (Black): Shows information about deleted files or searches.

Physical Location (Yellow): Geographic info from the system.

External Device/USB Usage (Blue): Tracks USBs or other devices used.

Account Usage (Gray): Login/logout and session activity.

Browser Usage (Orange): Tracks internet or browser activity.

Key idea: By looking at these artifacts, investigators can figure out what happened on a computer at a certain time.

**Digital Forensic Artifacts: NTFS Timestamps**

What it is:
Files on Windows NTFS systems have timestamps that show what happened to them and when. These timestamps help investigators track file activity.

The Four Key Timestamps (MACB):

Modified (M): Last time the file content was changed.

Accessed (A): Last time the file was opened or used.

Metadata Change (C): Last time file info changed (like name, permissions).

Created (B): When the file was first created.

How it helps:

These timestamps update in predictable ways.

For example, renaming a file only changes the Metadata Change time.

By looking at MACB times, investigators can see exactly what actions happened to files.

Key idea: Timestamps are a reliable way to see file activity and track events on a Windows system.

A Battle Against Time in Digital Forensics

The Problem:

Time works against investigators. Many digital artifacts are temporary or stored in limited space.

When storage limits are reached, old files may be deleted to make room for new ones.

Deleted files are marked for overwriting, so if the system needs space, the old data may be partially or fully erased.

Why it matters:

Partial or overwritten files may still provide clues.

Delays in detecting breaches (days, months, or years) make recovery harder.

Solutions for Investigators:

Use backups or restore points.

Recover deleted files using forensic tools ("data carving").

Act quickly, because the longer you wait, the harder it is to recover evidence.

Key idea: Digital evidence can disappear over time, so speed and proper recovery techniques are crucial.

**What DFIR is:**

DFIR (Digital Forensics and Incident Response) isn't just about disk images.

It combines multiple specialties to investigate incidents and find evidence.

Teams are diverse, with people skilled in different areas and using various tools.

Key DFIR Subdisciplines:

**Endpoint Forensics:**

Focuses on a computer (disk and memory).

Disk artifacts: files, logs, system info.

Memory artifacts: running processes, hidden malware.

**Network Forensics:**

Analyzes network traffic (like IP flows or full-packet captures).

Helps understand where data went and what happened on the network.

**Threat Intelligence (TI):**

Collects info on attackers: tools, methods, and targets (TTPs).

Guides investigations and predicts attacker behavior.

**Reverse Engineering:**

Examines malware or malicious tools to understand how they work.

Supports the team in understanding attacks and developing defenses.

Key idea: DFIR teams combine multiple skills and tools to fully investigate incidents, not just rely on one type of evidence.

**Digital Forensics Tools (Very Simple)**

What these tools do:
They help investigators collect, analyze, and understand digital evidence.

1. Open-Source Tools (Free)

The Sleuth Kit / Autopsy: Popular tools to analyze disks and files (command-line and GUI).

Custom scripts (Python, PowerShell, Perl): Used to solve special problems or analyze specific data.

Log2timeline / Plaso: Creates timelines showing what happened and when.

✅ Free and widely used by the community.

2. Standalone Virtual Machines (Ready-Made Systems)

SANS SIFT Workstation: A full forensic analysis system with many tools already installed.

REMnux: Used mainly for malware analysis.

✅ Easy to use and safe because they run in virtual machines.

3. Proprietary / Closed-Source Tools (Paid)

EnCase

X-Ways

✅ Professional tools used by companies and law enforcement, but they cost money and require licenses.

**Incident Handling Fundamentals (Very Simple)**

What is incident handling?
It is a plan for what to do when something bad happens, like:

Hacking or intrusions

Cyber theft

Denial-of-service attacks

Malware

Even fires or floods

Why it's important:

An incident will happen sooner or later.

It's not "if", it's "when."

Incidents can be intentional (attacks) or unintentional (mistakes, disasters).

Why you need a plan:

Incident plans are like car insurance:

You don't use it every day,

But when something bad happens, you're very glad you have it.

A plan tells you exactly what to do, so you don't panic or forget steps.

Key points:

Just watching an attack is not enough — you must take action.

Good documentation helps reduce mistakes during stressful situations.

Ignoring incidents is dangerous and can destroy a company over time.

Main Idea:

Planning is everything.
Having an incident response plan helps you act fast, reduce damage, and survive major security problems.

**Incident vs Event (Very Simple)**
What is an Event?

An event is anything that happens in a system or network.

Examples:

System starts (boot)

A user logs in

A system crash

Network traffic (packet flooding)

Events are normal and common.

What is an Incident?

An incident is an event (or group of events) that causes harm or tries to cause harm.

Harm means:

Data is changed, stolen, or deleted

Systems stop working

Security is broken

Incidents are dangerous events.

Key Differences:

All incidents are made of events

Not all events are incidents

The same event can be an incident or not, depending on the situation:

Authorized login → Event (normal)

Unauthorized login → Incident (harmful)

**Six-Step Incident Handling Process (Very Simple)**

This is a standard method used by many organizations.
Names may change, but the steps are almost always the same.

1. Preparation

Get ready before an incident happens

Policies, tools, monitoring, training

2. Identification (Detection)

Discover that something bad is happening

Alerts, logs, user complaints, IDS warnings

3. Containment

Stop the problem from spreading

Isolate infected systems

Often done in two steps:

Short-term: quick stop

Long-term: after collecting evidence

4. Eradication

Remove the threat

Delete malware, kill attacker processes

Reset passwords, clean systems

5. Recovery

Bring systems back to normal

Restore services and business operations

6. Lessons Learned

Review what happened

Fix weaknesses

Improve security to prevent it again

Main Idea:

Incident response follows clear steps to reduce damage, fix the problem, and prevent future attacks.

Easy way to remember (PICERL):

Prepare → Identify → Contain → Eradicate → Recover → Learn

**Common Problems in Incident Response (Very Simple)**

1. Poor Security Basics

Weak passwords

Too many user permissions

No proper monitoring or logs

Little or no threat intelligence

This makes attacks easy and hard to detect.

2. Poor Visibility

Organizations don't fully see what's happening in their systems

Missing logs and monitoring means attackers stay hidden

3. No Proper Scoping

Only fixing the systems they know are hacked

Not checking for other infected systems
This causes the attack to continue elsewhere.

4. Incomplete Containment

Acting too fast (killing malware immediately)

Evidence is lost

Attackers may still have access

5. Not Fixing the Real Problem

Cleaning systems but not fixing the weakness

Example: removing malware but keeping weak passwords

6. Ignoring Lessons Learned

Same attack happens again

No improvement in security

Deeper Problem:

Using a linear (one-time) approach

Incident response should be continuous and adaptive, not "do it once and forget it"

Main Idea:

Bad preparation and poor follow-up allow attackers to return again and again.

Need a More Dynamic Approach
One of the things to realize is that there is no "recipe" for incident response. One of the reasons is that
incidents are non-linear, that is, they don't happen as a single set of events. There are multiple events
happening (e.g., users browsing the web, accessing files, etc.) in the environment when an incident occurs.

**Preparation (Very Simple)**

What it is:
Preparation means getting ready before an incident happens.
This is the most important step, but it’s often skipped.

1. Know Your Organization

Know what systems and data are most important

Know the policies and procedures

Decide in advance:

When to call law enforcement

Who makes decisions during an incident

2. Have Good Visibility

You must see what’s happening in your systems

This includes:

Network logs

Host (computer) logs

Logs should go to a central place (SIEM)

Know:

Who reviews logs

How often

What gaps exist

Logs become the main evidence during investigations.

3. Plan System Rebuilding

Sometimes the best fix is rebuilding systems from scratch

This is often the fastest and safest way to remove rootkits and deep malware

4. Build the Right Team

Incident handlers must:

Work well under pressure

Work as a team

Make calm, realistic decisions

Not everyone is suited for incident response

5. Train and Prepare Checklists

Train regularly so skills don’t fade

Use checklists for:

Shutting down systems

Restoring systems

Keep checklists updated

Main Idea:

Good preparation saves time, reduces stress, and limits damage during incidents.

**Identification, Detection, Verification & Triage (Very Simple)**

1. Detection – How you notice something

Incidents can be detected from:

Network devices: firewall, IDS

Host systems: system and application logs

People: admins or users notice something is wrong

Threat intelligence feeds

Third parties: banks, vendors, or partners warning you

Anything suspicious can be a sign.

2. Verification – Is it real?

This is the first and most important decision:

Is this a real incident or a false alarm?

Sometimes it's obvious (website defaced)

Sometimes it needs deeper investigation or forensics

Don't panic or overreact too fast.

3. Triage – How serious is it?

After verification, decide:

What type of incident is it?

How big is the impact?

How fast do we need to respond?

This affects who is involved and how strong the response is.

4. Good Practices

Alert early, but stay calm

Avoid shutting down systems unless necessary

Assign one team leader to manage the response

Use the help desk to track events and communication

Main Idea:

First detect, then verify, then prioritize the incident so you respond correctly and avoid unnecessary damage.

**Containment (Very Simple)**

Goal:
Stop the attacker from continuing to work in your systems and spreading further.

Why containment matters:

Attackers often:

Move to other systems (lateral movement)

Try to stay hidden (persistence)

You must know how far the attack spread (proper scoping).

Common Containment Actions:

Isolate infected systems (remove from network)

Patch systems to close vulnerabilities

Remove attacker accounts or processes

Block traffic using firewalls or routers

Change DNS entries to stop malicious connections

Change passwords to prevent re-entry

Important Points:

Containment may happen in multiple phases:

Quick containment to stop damage

Evidence collection

Stronger containment later

Sometimes containment decisions are business decisions (resources, downtime).

Forensics Reminder:

If possible, take a forensically sound backup before making changes

Keep detailed notes of everything you do

Main Idea:

Containment is about quickly stopping the attack while preserving evidence and limiting damage.

What it means:
Eradication is about removing the attacker completely from the system.

Containment = stop the attacker

Eradication = remove the attacker

What happens in eradication:

Remove malicious files and processes

Delete attacker accounts

Restore systems from trusted backups

Fix the vulnerability that allowed the attack

Check for backdoors left by the attacker

Why this step is important:

Rebuilding a system without fixing the root cause can lead to reinfection

Attackers often leave hidden access (backdoors)

If the weakness isn't fixed, the attack will happen again

Extra steps:

Run a vulnerability scan after cleanup

Fix any new or old security gaps

Main Idea:

Eradication means clean everything AND fix why it happened, so the attack doesn't come back.

**Remediation (Very Simple)**

What it means:
Remediation is about fixing the real reason the incident happened — not just cleaning up.

Example:

Incident caused by a weak password

Containment: change the password

Remediation: ask why weak passwords were allowed

Weak policy?

No enforcement?

Poor user training?

Common root causes:

Poor user training

Bad or unclear incident response procedures

Ignoring threat intelligence

Users disabling security (like firewall turned off)

Flat networks that allow easy attacker movement

What remediation includes:

Fix all contributing factors, not just one system

Improve policies, controls, and network design

Update procedures and training

Ongoing monitoring:

Watch affected systems closely

Look for signs of return (rootkits or backdoors)

Monitor:

Network sensors

Host logs

Application logs

Main Idea:

Remediation means fix the root cause and watch carefully, so the attacker can't come back.

**Lessons Learned (Very Simple)**

What it is:

This step is about learning from the incident so it doesn’t happen again.

What to do:

Write a final report:

What happened

How it was handled

What worked and what didn’t

What needs to be improved

Why timing matters:

Right after the incident, people are motivated to fix things

Over time, that urgency fades

Do the review as soon as possible.

Team review:

Meet with everyone involved

Agree on:

Security improvements

Process changes

Follow-up reviews:

Check progress after:

30 days

60 days

90 days

Ask:

Were fixes implemented?

Did the same attack happen again?

**Key Mistakes in Incident Handling (Very Simple)**

Not reporting or asking for help

Ignoring the problem or trying to handle it alone

Incomplete or missing notes

Failing to document actions or observations

Mishandling or destroying evidence

Losing proof needed for investigation or legal purposes

No working backups

Systems can't be restored if backups don't exist

Failure to contain or eradicate

Attacker remains in the system, causing more damage

Failure to prevent reinfection

Same vulnerability exploited again

Failure to apply lessons learned

Same mistakes repeat; security doesn't improve

**Taking Notes (Very Simple)**

Why it matters:
Taking notes is critical during an incident or investigation because they:

Form the basis of your final report

Jog your memory for details later

Document damages (useful for insurance or legal purposes)

Help you stay organized and methodical

How to take notes:

Record everything so someone else with similar skills can recreate your work

Focus on facts, not opinions

Avoid notes that show bias or look unprofessional

Hand-written notes are often seen as more reliable than digital ones

Extra benefits:

Helps slow you down and avoid mistakes

Provides a clear record of what happened, what you did, and why

Main Idea:

Good notes = better reports, clearer evidence, and safer investigations.

**Putting the Steps Together (Incident Response)**

Key Points:

Customize steps for your organization's environment (network, policies, systems)

Every incident is different – no one-size-fits-all

Planning is everything – make checklists and tested procedures

Practice often – simulations, penetration tests, and drills help

Incident Response Steps Recap:

Preparation: Plan ahead; incidents will happen

Identification: Detect and verify if an event is truly an incident

Containment: Stop the attack from spreading and gather evidence

Eradication: Remove the threat and any malicious artifacts

Recovery: Restore systems fully before returning to production

Remediation: Fix the root cause and monitor for return attacks

Lessons Learned: Review, report, and improve processes

Threat Hunting

Proactive approach: Search for attacks or vulnerabilities before they're detected

Uses knowledge of attacker behavior and past incidents

Can uncover hidden or historical attacks

Results usually feed back into forensics and incident response